BASIC LINUX COMMANDS

EX. NO. 1a **WORKING WITH FILES AND DIRECTORIES**

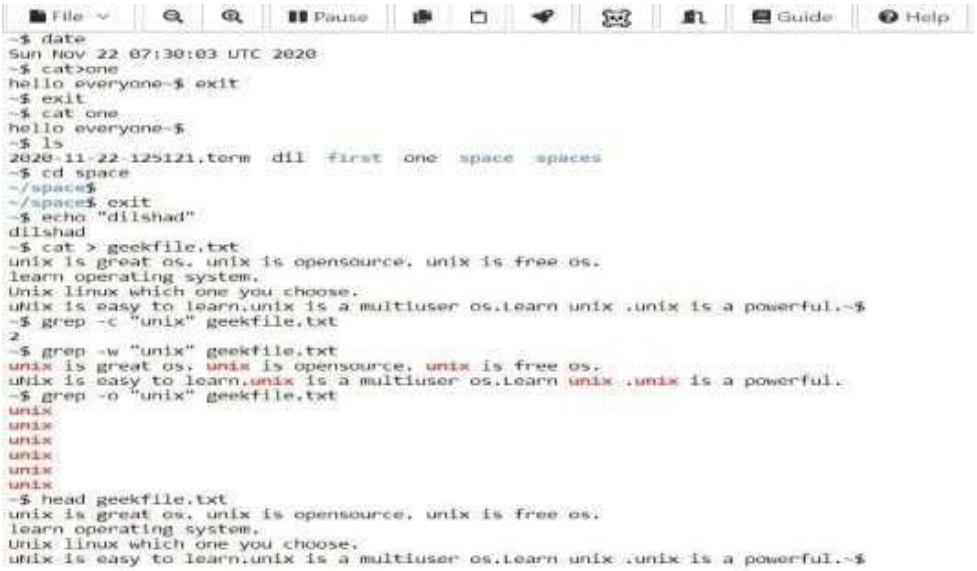
AIM:

To study the UNIX commands for accessing files and directories.

COMMANDS DESCRIPTION:

- cat --- for creating and displaying short files
- chmod --- change permissions
- cd --- change directory
- cp --- for copying files
- date --- display date
- echo --- echo argument
- ftp --- connect to a remote machine to download or upload files
- grep --- search file
- head --- display first part of file
- ls --- see what files you have
- lpr --- standard print command
- more --- use to read files
- mkdir --- create directory
- mv --- for moving and renaming files
- ncftp --- especially good for downloading files via anonymous ftp.
- print --- custom print command
- pwd --- find out what directory you are in
- rm --- remove a file
- rmdir --- remove directory
- rsh --- remote shell
- setenv --- set an environment variable
- sort --- sort file
- tail --- display last part of file
- tar --- create an archive, add or extract files
- telnet --- log in to another machine
- wc --- count characters, words, lines

EXECUTION & OUTPUT:



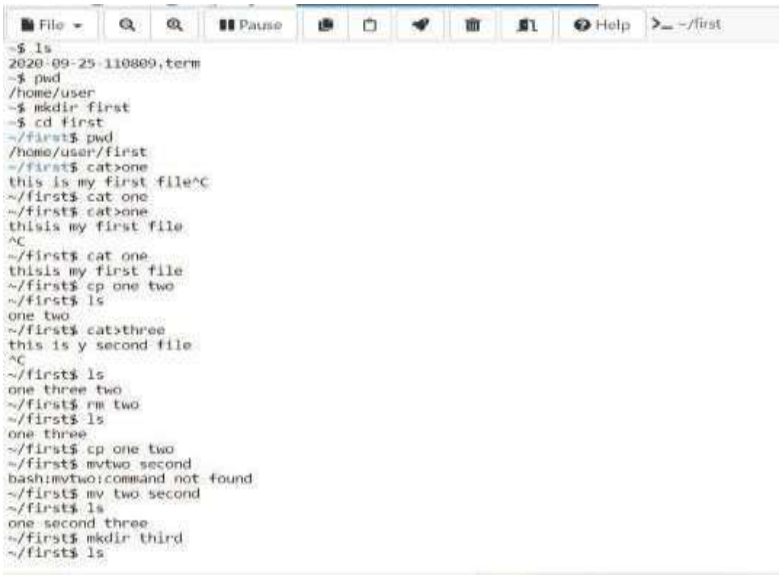
```
File  Pause  Guide  Help
~$ date
Sun Nov 22 07:30:03 UTC 2020
~$ cat>one
hello everyone~$ exit
~$ exit
~$ cat one
hello everyone~$
~$ ls
2020-11-22-125121.term  dil  first  one  space  spaces
~$ cd space
~/space$
~/space$ exit
~$ echo "dilshad"
dilshad
~$ cat > geekfile.txt
unix is great os. unix is opensource. unix is free os.
learn operating system.
Unix linux which one you choose.
uNix is easy to learn.unix is a multiuser os.Learn unix .unix is a powerful.~$
~$ grep -c "unix" geekfile.txt
2
~$ grep -w "unix" geekfile.txt
unix is great os. unix is opensource. unix is free os.
uNix is easy to learn.unix is a multiuser os.Learn unix .unix is a powerful.
~$ grep -o "unix" geekfile.txt
unix
unix
unix
unix
unix
unix
~$ head geekfile.txt
unix is great os. unix is opensource. unix is free os.
learn operating system.
Unix linux which one you choose.
uNix is easy to learn.unix is a multiuser os.Learn unix .unix is a powerful.~$
```



```
File  Pause  Guide  Help
learn operating system.
Unix linux which one you choose.
uNix is easy to learn.unix is a multiuser os.Learn unix .unix is a powerful.~$
~$ grep -c "unix" geekfile.txt
2
~$ grep -w "unix" geekfile.txt
unix is great os. unix is opensource. unix is free os.
uNix is easy to learn.unix is a multiuser os.Learn unix .unix is a powerful.
~$ grep -o "unix" geekfile.txt
unix
unix
unix
unix
unix
unix
~$ head geekfile.txt
unix is great os. unix is opensource. unix is free os.
learn operating system.
Unix linux which one you choose.
uNix is easy to learn.unix is a multiuser os.Learn unix .unix is a powerful.~$
~$ lpr geeksfile.txt
lpr: Error - unable to access "geeksfile.txt" - No such file or directory
~$ lpr myfile.txt
lpr: Error - unable to access "myfile.txt" - No such file or directory
~$ mkdir first
mkdir: cannot create directory 'first': File exists
~$ ls
2020-11-22-125121.term  dil  first  geekfile.txt  one  space  spaces
~$ pwd
/home/user
~$ rm one
~$ ls
2020-11-22-125121.term  dil  first  geekfile.txt  space  spaces
~$ rmdir first
~$ ls
2020-11-22-125121.term  dil  geekfile.txt  space  spaces
```


File
Pause
~$ rmdir first
~$ ls
2020-11-22-125121.term dil geekfile.txt space spaces
~$ cat > file.txt
abhishek
chitransh
satish
rajan
naveen
divyam
harsh~$
~$ sort file.txt
abhishek
chitransh
divyam
harsh
naveen
rajan
satish
~$ cat state.txt
cat: state.txt: No such file or directory
~$ cat>state.txt
Andhra Pradesh
Arunachal Pradesh
Assam
Bihar
Chhattisgarh~$
~$ wc state.txt
4 7 57 state.txt
~$ tail state.txt
Andhra Pradesh
Arunachal Pradesh
Assam
Bihar
Chhattisgarh~$
~$


File
Pause
Help
~/one/two
~$ date
Fri Aug 28 06:21:06 UTC 2020
~$ echo hai
hai
~$ man echo
~$ mkdir one
~$ ls
2020-08-28-115056.term one
~$ cd one
~/one$ nano dil
~/one$ cat dil
iam dilshad
2nd year student
~/one$ nano dil
~/one$ cat dil
iam dilshad
2nd year student
operating systems
~/one$ mkdir two
~/one$ ls
dil two
~/one$ cd two
~/one/two$ nano patan
~/one/two$
~/one/two$ cat patan
Operating systems class is held.
It was a lab session.
~/one/two$ touch dil
~/one/two$ ls -l
total 2
-rw-r--r-- 1 user user 0 Aug 28 06:53 dil
-rw-r--r-- 1 user user 55 Aug 28 06:39 patan
~/one/two$ chmod o+wx dil



Result: The general purpose utility commands was executed successfully.

Assessment of an Individual experiment	
Algorithm	
Program	
Output	
Viva	

Ex. No. 1b **General Purpose Utility Commands**

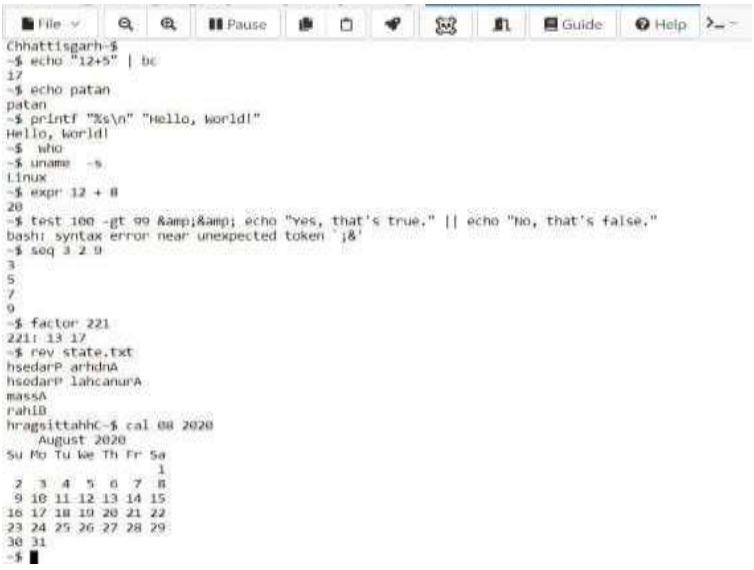
AIM:

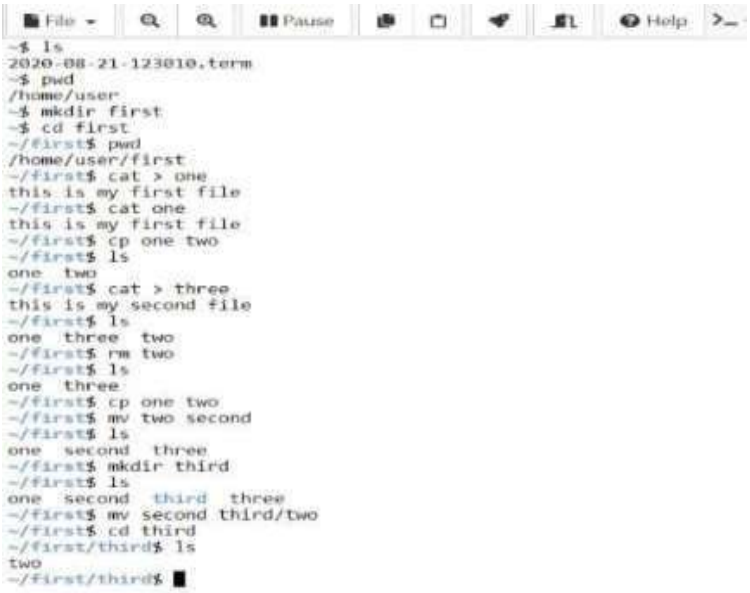
To work with some of the general purpose utility commands in UNIX.

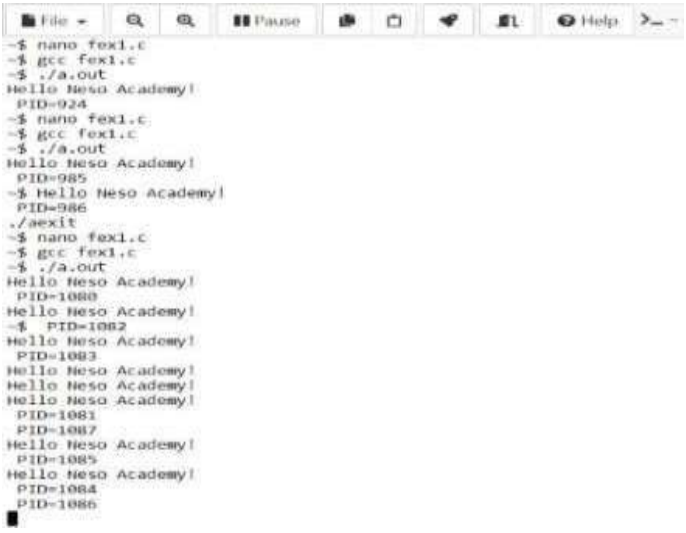
COMMANDS DESCRIPTION:

- date - print or set the system date and time
- bc - An arbitrary precision calculator language
- echo - Display a line of text.
- printf - Format and print data
- passwd - update user's authentication tokens
- who - Show who is logged on
- who am I - Show who is logged on and what they are doing
- uname - Print system information
- expr - Evaluate expressions
- test - Check file types and compare values
- seq - Print a sequence of numbers
- factor - Factor numbers
- rev - reverse lines of a file or files
- cal - Displays a calendar
- date - print or set the system date and time
- bc - An arbitrary precision calculator language
- echo - Display a line of text.
- printf - Format and print data
- passwd - update user's authentication tokens
- who - Show who is logged on
- w - Show who is logged on and what they are doing
- uname - Print system information
- expr - Evaluate expressions
- test - Check file types and compare values
- seq - Print a sequence of numbers
- factor - Factor numbers
- rev - reverse lines of a file or files

EXECUTION & OUTPUT:







Result : The general purpose utility commands was executed successfully.

Assessment of an Individual experiment	
Algorithm	
Program	
Output	
Viva	

SYSTEM CALLS

Ex. No. 2a **Fork System Call**

AIM:

To write a program to create a child process using the system calls –fork.

ALGORITHM:

1. fork() to create a child process from the main/parent process

The fork() system function does not accept any argument. It returns an integer of the type pid_t.

On success, fork() returns the PID of the child process which is greater than 0. Inside the child process, the return value is 0. If fork() fails, then it returns -1.

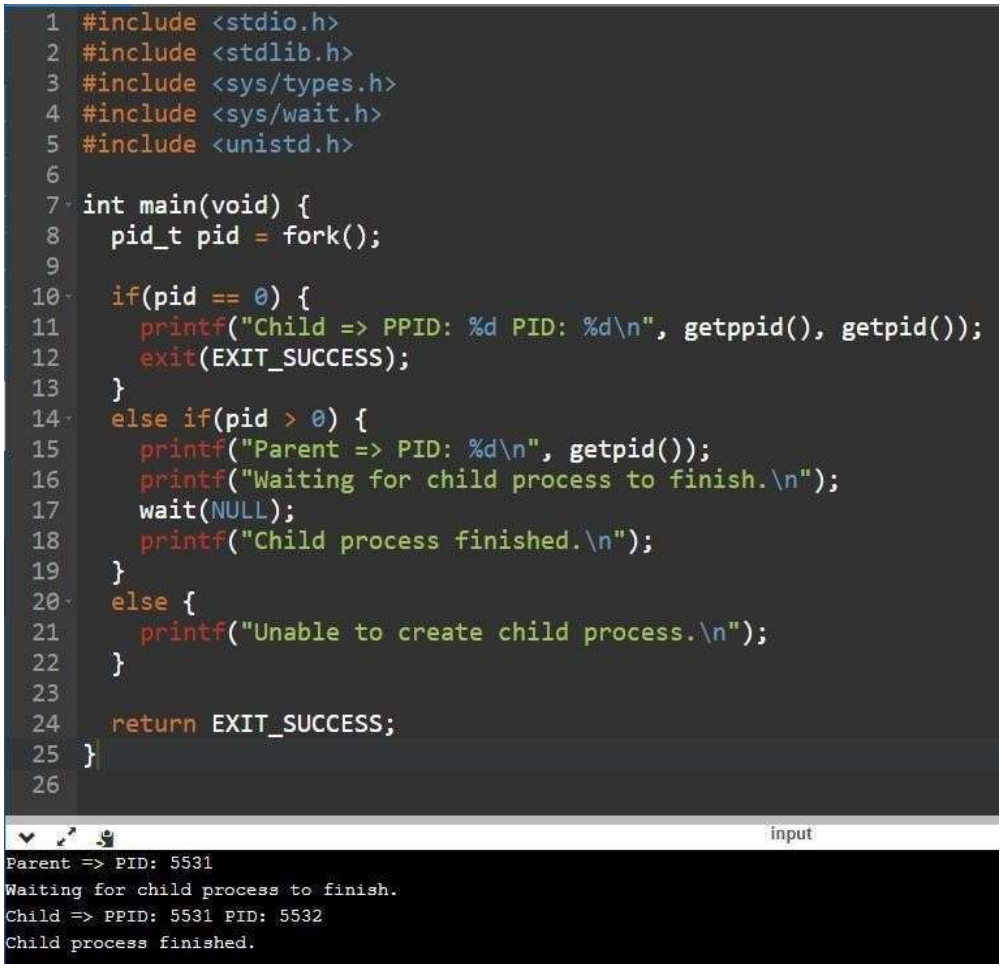
2. print the PID (Process ID) and PPID (Parent Process ID) from child and parent process.
3. On the parent process wait(NULL) is used to wait for the child process to finish
4. On the child process, exit() is used to finish the child process.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include  <unistd.h>
int main(void) {
  pid_t pid = fork();
  if(pid == 0) {
    printf("Child => PPID: %d PID: %d\n", getppid(), getpid());
    exit(EXIT_SUCCESS);
  }
  else if(pid > 0) {
```

```
    printf("Parent => PID: %d\n", getpid());
    printf("Waiting for child process to finish.\n");
    wait(NULL);
    printf("Child process finished.\n");
  }
  else {
    printf("Unable to create child process.\n");
  }
  return EXIT_SUCCESS;
}
```

EXECUTION & OUTPUT:



Result : Hence create a child process using the system calls – fork executed successfully

Assessment of an Individual experiment	
Algorithm	
Program	
Output	
Viva	

Ex. No. 2b **EXEC System Call**

AIM:

To write a program to display time and date using exec system call.

ALGORITHM

1.Current process image is overwritten with a new process image.

2.New process image is the one you passed as exec argument

3.The currently running process is ended

4. New process image has same process ID, same environment, and same file descriptor (because process is not replaced process image is replaced)

5. The CPU stat and virtual memory is affected. Virtual memory mapping of the current process image is replaced by virtual memory of new process image.

6. execlp("date","date", NULL); //uses the PATH to find date, try: %echo $PATH

PROGRAM:

```
#include <stdio.h>

#include <unistd.h>

#include <stdlib.h>

int main(int argc, char *argv[])

{

   printf("PID of example.c = %d\n", getpid());

   char *args[] = {"Hello", "C", "Programming", NULL};

   execlp("date","date", NULL);

   execv("./hello", args);

   printf("Back to example.c");

   return 0;

}
```

```c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
int main(int argc, char *argv[])
{
    printf("PID of example.c = %d\n", getpid());
    char *args[] = {"Hello", "C", "Programming", NULL};
    execlp("date","date", NULL);
    execv("./hello", args);
    printf("Back to example.c");
    return 0;
}
```

input

```
PID of example.c = 1538
Mon Apr 26 05:28:02 UTC 2021
```

Result: Thus to display time and date using exec system call was executed successfully.

Assessment of an Individual experiment	
Algorithm	
Program	
Output	
Viva	

Ex.No. 2c **STAT System Call**

DESCRIPTION:

stat() is a Unix system call that returns file attributes about an inode. The semantics of stat() vary between operating systems. As an example, Unix command ls uses this system call to retrieve information on files that includes:

- atime: time of last access (ls -lu)
- mtime: time of last modification (ls -l)
- ctime: time of last status change (ls -lc)

AIM:

To write a program to implement STAT system call.

ALGORITHM:

These functions return information about a file, in the buffer

pointed to by statbuf. No permissions are required on the file

itself, butin the case of stat(), fstatat(), and lstat()execute

(search) permission is required on all of the directories in pathname

that lead to the file.

stat() and fstatat() retrieve information about the file pointed to

by pathname; the differences for fstatat() are described below.

lstat() is identical to stat(), except that if pathname is a symbolic

link, then it returns information about the link itself, not the file

that the link refers to.

fstat() is identical to stat(), except that the file about which information is to be retrieved is specified by the file descriptor fd.

The stat structure

All of these system calls return a stat structure, which contains the following fields:

```
struct stat {
    dev_t     st_dev;         /* ID of device containing file */
    ino_t     st_ino;         /* Inode number */
    mode_t    st_mode;        /* File type and mode */
    nlink_t   st_nlink;       /* Number of hard links */
    uid_t     st_uid;         /* User ID of owner */
    gid_t     st_gid;         /* Group ID of owner */
    dev_t     st_rdev;        /* Device ID (if special file) */
    off_t     st_size;        /* Total size, in bytes */
    blksize_t st_blksize;     /* Block size for filesystem I/O */
    blkcnt_t  st_blocks;      /* Number of 512B blocks allocated */
```

```
        /* Since Linux 2.6, the kernel supports nanosecond
           precision for the following timestamp fields.
           For the details before Linux 2.6, see NOTES. */

        struct timespec st_atim;  /* Time of last access */
        struct timespec st_mtim; /* Time of last modification */
        struct timespec st_ctim;  /* Time of last status change */

    #define st_atime st_atim.tv_sec      /* Backward compatibility */
    #define st_mtime st_mtim.tv_sec
    #define st_ctime st_ctim.tv_sec
    }
```

PROGRAM:

```
#include <sys/types.h>
#include <sys/stat.h>
#include <time.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
   struct stat sb;

   if (argc != 2) {
      fprintf(stderr, "Usage: %s <pathname>\n", argv[0]);
      exit(EXIT_FAILURE);
```

```
}

if (stat(argv[1], &sb) == -1) {
    perror("stat");
    exit(EXIT_FAILURE);
}

printf("File type:                ");

switch (sb.st_mode & S_IFMT) {
    case S_IFBLK:  printf("block device\n");            break;
    case S_IFCHR:  printf("character device\n");        break;
    case S_IFDIR:  printf("directory\n");               break;
    case S_IFIFO:  printf("FIFO/pipe\n");               break;
    case S_IFLNK:  printf("symlink\n");                 break;
    case S_IFREG:  printf("regular file\n");            break;
    case S_IFSOCK: printf("socket\n");                  break;
    default:       printf("unknown?\n");                break;
}

printf("I-node number:            %ld\n", (long) sb.st_ino);

printf("Mode:                     %lo (octal)\n",
    (unsigned long) sb.st_mode);

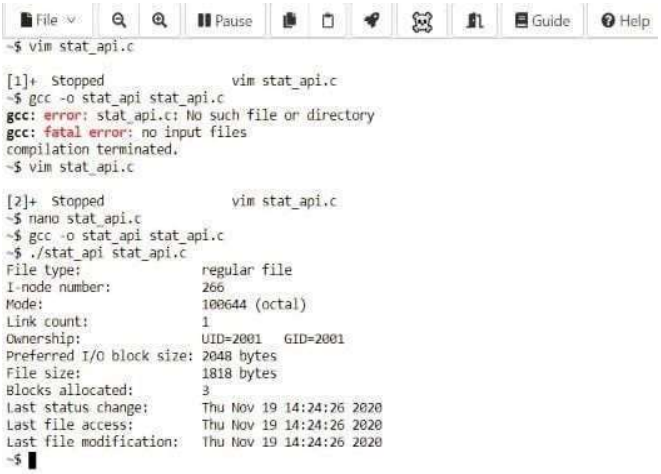
printf("Link count:               %ld\n", (long) sb.st_nlink);
printf("Ownership:                UID=%ld  GID=%ld\n",
    (long) sb.st_uid, (long) sb.st_gid);
```

```
    printf("Preferred I/O block size: %ld bytes\n",
        (long) sb.st_blksize);
    printf("File size:                %lld bytes\n",
        (long long) sb.st_size);
    printf("Blocks allocated:         %lld\n",
        (long long) sb.st_blocks);

    printf("Last status change:       %s", ctime(&sb.st_ctime));
    printf("Last file access:         %s", ctime(&sb.st_atime));
    printf("Last file modification:  %s", ctime(&sb.st_mtime));

    exit(EXIT_SUCCESS);
}
```

EXECUTION & OUTPUT:



Result: Thus program to implement STAT system call was implemented successfully.

Assessment of an Individual experiment	
Algorithm	
Program	
Output	
Viva	

Ex. No. 2d **Wait System Call**

DESCRIPTION:

The system call wait() is blocks the calling process until one of its child processes exits or a signal is received. For our purpose, we shall ignore signals. wait() takes the address of an integer variable and returns the process ID of the completed process. Some flags that indicate the completion status of the child process are passed back with the integer pointer. One of the main purposes of wait() is to wait for completion of child processes.

The execution of wait() could have two possible situations.

1. If there are at least one child processes running when the call to wait() is made, the caller will be blocked until one of its child processes exits. At that moment, the caller resumes its execution.
2. If there is no child process running when the call to wait() is made, then this wait() has no effect at all. That is, it is as if no wait() is there.

AIM:

To write a program using wait system call

ALGORITHM:

1.print before fork. Then fork() system call creates a child process.

2.wait() system call is added to the parent section of the code.

3. Hence, the moment processor starts processing the parent, the parent process is suspended because the very first statement is wait(NULL).

4. Thus, first, the child process runs, and the output lines are all corresponding to the child process. Once the child process finishes, parent resumes and prints all its printf() statements.

5. The NULL inside the wait() means that we are not interested to know the status of change of state of child process

PROGRAM:

```
#include<unistd.h>

#include<sys/types.h>

#include<stdio.h>
```

```
#include<sys/wait.h>
int main()
{
pid_t p;
printf("before fork\n");
p=fork();
if(p==0)//child
{
printf("I am child having id %d\n",getpid());
printf("My parent's id is %d\n",getppid());
}
else//parent
{
wait(NULL);
printf("My child's id is %d\n",p);
printf("I am parent having id %d\n",getpid());
}
printf("Common\n");
}
```

EXECUTION & OUTPUT:

```c
#include<unistd.h>
#include<sys/types.h>
#include<stdio.h>
#include<sys/wait.h>
int main()
{
pid_t p;
printf("before fork\n");
p=fork();
if(p==0)//child
{
printf("I am child having id %d\n",getpid());
printf("My parent's id is %d\n",getppid());
}
else//parent
{
wait(NULL);
printf("My child's id is %d\n",p);
printf("I am parent having id %d\n",getpid());
}
printf("Common\n");
}
```

```
before fork
I am child having id 401
My parent's id is 400
Common
My child's id is 401
I am parent having id 400
Common
```

Assessment of an Individual experiment	
Algorithm	
Program	
Output	
Viva	

Result : Thus program using wait system call was executed successfully

Ex. No. 2e **Input Output system Call**

DESCRIPTION:

- creat(name, permissions) – Used to create a file with the name and mode specified. Here, permission would be a number. 0666 means read write permissions.

- open(name, mode) – Used to open a file name in the mode (read or write) specified. 0 is for opening in read mode, 1 for writing and 2 for both.

- close(fd) – Close a opened file.

- unlink(fd) – Delete a file.

- read(fd, buffer, n_to_read) – Read data from a file.

- write(fd, buffer, n_to_write) - write data from to a file.

- lseek(fd, offest, whence) - Move the read/write pointer to the specified location.

AIM:

To write a program to implement the concept of I/O call.

ALGORITHM:

1.Create new empty file on disk

2.Create file table entry

3.Set first unused file descriptor to point to file table entry

4.Return file descriptor used, -1 upon failure

5. open: Used to Open the file for reading, writing or both.

Syntax in C language

#include<sys/types.h>

#include<sys/stat.h>

#include <fcntl.h>

6. int open (const char* Path, int flags [, int mode]);

Parameters

Path : path to file which you want to use

use absolute path begin with /, when you are not work in same directory of file.

Use relative path which is only file name with extension, when you are work in same directory of file.

flags : How you like to use

O_RDONLY: read only, O_WRONLY: write only, O_RDWR: read and write, O_CREAT: create file if it doesnt exist, O_EXCL: prevent creation if it already exists

How it works in OS

7.Find existing file on disk

8.Create file table entry

9.Set first unused file descriptor to point to file table entry

10.Return file descriptor used, -1 upon failure

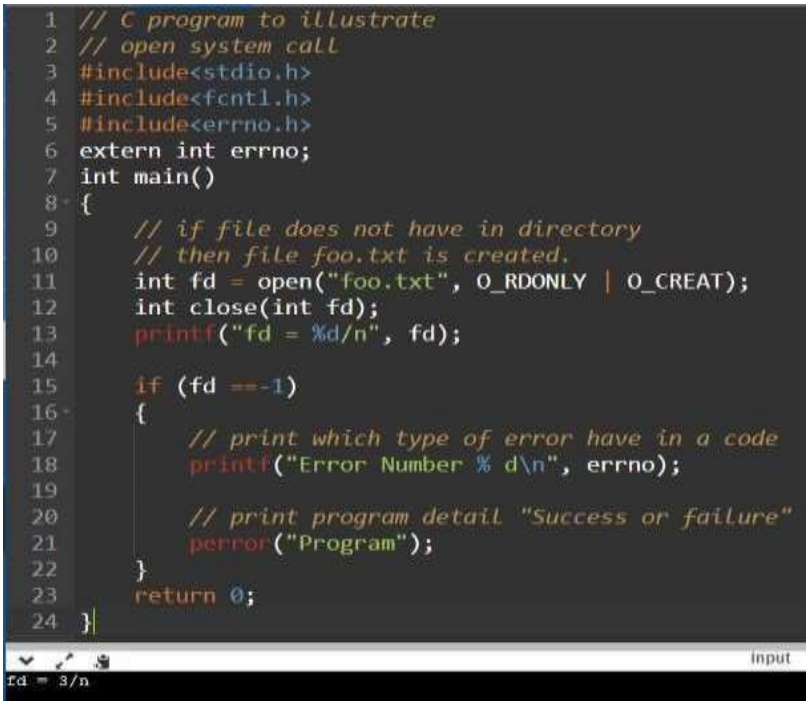
PROGRAM:

```
// C program to illustrate
// open system call
#include<stdio.h>
#include<fcntl.h>
#include<errno.h>
extern int errno;
int main()
{
        // if file does not have in directory
        // then file foo.txt is created.
        int fd = open("foo.txt", O_RDONLY | O_CREAT);
        int close(int fd);
        printf("fd = %d/n", fd);
```

```
    if (fd ==-1)
    {
        // print which type of error have in a code
        printf("Error Number % d\n", errno);

        // print program detail "Success or failure"
        perror("Program");
    }
    return 0;
}
```

EXECUTION & OUTPUT:



Result : Thus program to implement the concept of I/O call was executed successfully.

Assessment of an Individual experiment	
Algorithm	
Program	
Output	
Viva	

SCHEDULING ALGORITHMS

Ex. No. 3a First Come First Serve Scheduling

AIM:

To write a program using C in UNIX environment to implement the first come first serve scheduling.

ALGORITHM:

Step1: Create the number of process.

Step2: Get the ID and Service time for each process.

Step3: Initially, Waiting time of first process is zero and Total time for the first process is the starting time of that process.

Step4: Calculate the Total time and Processing time for the remaining processes. Step5: Waiting time of one process is the Total time of the previous process.

Step6: Total time of process is calculated by adding Waiting time and Service time. Step7: Total waiting time is calculated by adding the waiting time for lack process. Step8: Total turn around time is calculated by adding all total time of each process.

Step9: Calculate Average waiting time by dividing the total waiting time by total number of process.

Step10: Calculate Average turn around time by dividing the total time by the number of process.

Step11: Display the result

PROGRAM:

```
#include<stdio.h>

struct process

{

int id,wait,ser,tottime;

}p[20];

main()

{
```

```
int i,n,j,totalwait=0,totalser=0,avturn,avwait;
printf("Enter number of process:");
scanf("%d",&n);
for(i=1;i<=n;i++)
{
printf("Enter process_id:");
scanf("%d",&p[i].id);
printf("Enter process service time:");
scanf("%d",&p[i].ser);
}
p[1].wait=0;
p[1].tottime=p[1].ser;
for(i=2;i<=n;i++)
{
for(j=1;j<i;j++)
{
p[i].wait=p[i].wait+p[j].ser;
}
totalwait=totalwait+p[i].wait;
p[i].tottime=p[i].wait+p[i].ser;
totalser=totalser+p[i].tottime;
}
avturn=totalser/n;
avwait=totalwait/n;
printf("Id\tservice\twait\ttotal");
for(i=1;i<=n;i++)
{
printf("\n%d\t%d\t%d\t%d\n",p[i].id,p[i].ser,p[i].wait,p[i].tottime);
```

```
}
printf("average waiting time %d\n",avwait);
printf("average turnaround time %d\n",avturn);
}
```

EXECUTION & OUTPUT:

```
Enter number of process:4
Enter process_id:901
Enter process service time:4
Enter process_id:902
Enter process service time:3
Enter process_id:903
Enter process service time:5
Enter process_id:904
Enter process service time:2
Id      service wait    total
901     4       0       4

902     3       4       7

903     5       7       12

904     2       12      14
average waiting time 5
average turnaround time 8
```

Result: Thus the C program to implement CPU scheduling algorithm for first come first serve was executed successfully

Assessment of an Individual experiment	
Algorithm	
Program	
Output	
Viva	

Ex. No. 3b **Shortest Job First Scheduling**

AIM:

To write a program using C in UNIX environment to implement the shortest job next concept.

ALGORITHM:

Step1:Get the number of process.

Step2:Get the id and service time for each process.

Step3:Initially the waiting time of first short process as 0 and total time of first short is process the service time of that process.

Step4:Calculate the total time and waiting time of remaining process.

Step5:Waiting time of one process is the total time of the previous process.

Step6:Total time of process is calculated by adding the waiting time and service time of each process.

Step7:Total waiting time calculated by adding the waiting time of each process.

Step8:Total turn around time calculated by adding all total time of each process.

Step9:calculate average waiting time by dividing the total waiting time by total number of process.

Step10:Calculate average turn around time by dividing the total waiting time by total number of process.

Step11:Display the result

PROGRAM:

```
#include<stdio.h>

struct ff

{

int pid,ser,wait;

}p[20];
```

```
struct ff tmp;
main()
{
int i,n,j,tot=0,avwait,totwait=0,tturn=0,aturn;
printf("enter the number of process");
scanf("%d",&n);
for(i=0;i<n;i++)
{
printf("enter process id");
scanf("%d",&p[i]);
printf("enter service time");
scanf("%d",&p[i].ser); p[i].wait=0;
}
for(i=0;i<n-1;i++)
{
for(j=i+1;j<n;j++)
{
if(p[i].ser>p[j].ser)
{
tmp=p[i]; p[i]=p[j]; p[j]=tmp;
}
}
}
printf("PID\tSER\tWAIT\tTOT\n");
for(i=0;i<n;i++)
{
tot=tot+p[i].ser;
```

```
tturn=tturn+tot;
p[i+1].wait=tot;
totwait=totwait+p[i].wait;
printf("%d\t%d\t%d\t%d\n",p[i].pid,p[i].ser,p[i].wait,tot);
}
avwait=totwait/n;
aturn=tturn/n;
printf("TOTAL WAITING TIME :%d\n",totwait);
printf("AVERAGE WAITING TIME : %d\n",avwait);
printf("TOTAL TURNAROUND TIME :%d\n",tturn);
printf("AVERAGE TURNAROUND TIME:%d\n",aturn);
}
```

EXECUTION & OUTPUT:

```
Enter the number of process:4
Enter process id:701
Enter service time:6
Enter process id:702
Enter service time:4
Enter process id:703
Enter service time:8
Enter process id:704
Enter service time:1
PID     SER     WAIT    TOT
704     1       0       1
702     4       1       5
701     6       5       11
703     8       11      19
TOTAL WAITING TIME :17
AVERAGE WAITING TIME : 4
TOTAL TURNAROUND TIME :36
AVERAGE TURNAROUND TIME:9
```

Result : Thus the C program to implement the CPU scheduling algorithm for shortest job first was executed successfully.

Ex. No. 3c **Round Robin Scheduling**

AIM:

To write a program using C in UNIX environment to implement Round-robin scheduling concept.

ALGORITHM:

Step 2: Receive inputs from the user to fill process id,burst time and arrival time.

Step 3: Calculate the waiting time for all the process id.

i) The waiting time for first instance of a process is calculated as:

a[i].waittime=count + a[i].arrivt

ii) The waiting time for the rest of the instances of the process is

calculated as:

a) If the time quantum is greater than the remaining burst time then

waiting time is calculated as:

a[i].waittime=count + tq

b) Else if the time quantum is greater than the remaining burst

time then waiting time is calculated as:

a[i].waittime=count - remaining burst time

Step 4: Calculate the average waiting time and average turnaround time

Step 5: Print the results of the step 4.

PROGRAM:

```
#include <stdio.h>
struct roundRobin
{
int pburst,pburst1,wtime,endtime,arrivt,boolean,flagcntl; char pname[5];
}a[5];
```

```
int  n,tq;
void input();
void initialize();
void calculate();
void display_waittime();
int main()
{
input();
initialize();
calculate();
display_waittime();
//getch();
//return 0;
}
void input()
{
int i;
printf("Enter Total no. of processes\n");
scanf("%d",&n);
for(i=0;i<n;i++)
{
printf("Enter process name:");
scanf("%s",&a[i].pname);
printf("Enter process burst time:");
scanf("%d",&a[i].pburst);
printf("Enter process arrival time:");
scanf("%d",&a[i].arrivt);
}
```

```
printf("\nEnter the time quantum/Time Slice:");
scanf("%d",&tq);
}

void initialize()
{ int i;

for(i=0;i<n;i++)
{
a[i].pburst1=a[i].pburst;
a[i].wtime=0;
a[i].endtime=0;
a[i].boolean=0;
a[i].flagcntl=0;
}
}

void calculate()
{
   int i,j=0,k=0,flag=1,count=0;
   printf("\n---GANTT CHART---\n");
   printf("0 | ");
while(flag)
{
for(i=0;i<n;i++)
{
if((k<n)&&(a[i].arrivt<=count)&&(a[i].flagcntl==0)) //calculating waiting time for first time
{
```

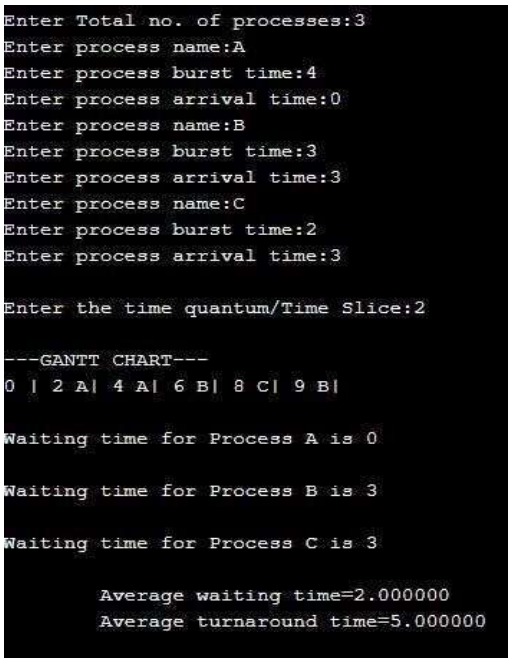
```
a[i].wtime=count-a[i].arrivt;
a[i].endtime=count;
a[i].boolean=1;
a[i].flagcntl=1;
k++;
}
if((a[i].pburst1>tq)&&(a[i].arrivt<=count))
{
if(a[i].boolean==1)
a[i].boolean=0;
else
a[i].wtime=a[i].wtime+(count-a[i].endtime);
count=count+tq;
a[i].pburst1=a[i].pburst1-tq; a[i].endtime=count;
printf("%d %s| ",count,a[i].pname);
}
else if((a[i].pburst1>0) && (a[i].pburst1<=tq) && (a[i].arrivt<=count))
{
if(a[i].boolean==1)
a[i].boolean=0;
else
a[i].wtime=a[i].wtime+(count-a[i].endtime);
count=count+a[i].pburst1;
a[i].endtime=count;
printf("%d %s| ",count,a[i].pname);
a[i].pburst1=0;
j++;
}
```

```
else if(j==n) flag=0;
}//end of for loop
}//end of while loop
}

void display_waittime()
{
   int i,tot=0,turn=0;

for(i=0;i<n;i++)
{
printf("\n\nWaiting time for Process %s is %d",a[i].pname,a[i].wtime);
tot=tot+a[i].wtime;
turn=turn+a[i].endtime-a[i].arrivt;
}
printf("\n\n\tAverage waiting time=%f",(float)tot/(float)n);
printf("\n\tAverage turnaround time=%f\n",(float)turn/(float)n);
}
```

EXECUTION & OUTPUT:



Result: Thus the C program to simulate CPU scheduling algorithm for round robin was executed successfully.

Assessment of an Individual experiment	
Algorithm	
Program	
Output	
Viva	

Ex.No. 3d **Priority Scheduling**

AIM:

To implement the priority scheduling using C in UNIX environment.

ALGORITHM:

Step 1: Inside the structure declare the variables.

Step 2: Declare the variable i,j as integer, totwtime and totttime is equal to zero.

Step 3: Get the value of „n‟ assign p and allocate the memory.

Step 4: Inside the for loop get the value of burst time and priority.

Step 5: Assign wtime as zero .

Step 6: Check p[i].pri is greater than p[j].pri .

Step 7: Calculate the total of burst time and waiting time and assign as turnaround time.

Step 8: Stop the program

PROGRAM:

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
   int et[20],at[10],n,i,j,temp,p[10],st[10],ft[10],wt[10],ta[10];
   int totwt=0,totta=0;
   float awt,ata;
   char pn[10][10],t[10];
   //clrscr();
   printf("Enter the number of process:");
   scanf("%d",&n);
```

```
for(i=0; i<n; i++)
{
   printf("Enter process name,arrivaltime,execution time & priority:");
   //flushall();
   scanf("%s%d%d%d",pn[i],&at[i],&et[i],&p[i]);
}for(i=0; i<n; i++)
   for(j=0; j<n; j++)
   {
      if(p[i]<p[j])
      {
         temp=p[i];
         p[i]=p[j];
         p[j]=temp;
         temp=at[i];
         at[i]=at[j];
         at[j]=temp;
         temp=et[i];
         et[i]=et[j];
         et[j]=temp;
         strcpy(t,pn[i]);
         strcpy(pn[i],pn[j]);
         strcpy(pn[j],t);
      }
   }
for(i=0; i<n; i++)

{
```

```
        if(i==0)
        {
            st[i]=at[i];
            wt[i]=st[i]-at[i];
            ft[i]=st[i]+et[i];
            ta[i]=ft[i]-at[i];
        }
        else
        {
            st[i]=ft[i-1];
            wt[i]=st[i]-at[i];
            ft[i]=st[i]+et[i];
            ta[i]=ft[i]-at[i];
        }
        totwt+=wt[i];
        totta+=ta[i];
    }
    awt=(float)totwt/n;
    ata=(float)totta/n;
    printf("\nPname\tarrivaltime\texecutiontime\tpriority\twaitingtime\ttatime");
    for(i=0; i<n; i++)
        printf("\n%s\t%5d\t\t%5d\t\t%5d\t\t%5d\t\t%5d",pn[i],at[i],et[i],p[i],wt[i],ta[i]);
    printf("\nAverage waiting time is:%f",awt);
    printf("\nAverage turnaroundtime is:%f",ata);
    getch();
}
```

EXECUTION & OUTPUT:

```
Enter the number of process:3
Enter process name,arrivaltime,execution time & priority:1 2 1 3
Enter process name,arrivaltime,execution time & priority:3 5 4 1
Enter process name,arrivaltime,execution time & priority:3 4 6 2

Pname   arrivaltime     executiontime   priority        waitingtime     tatime
3           5               4               1               0               4
3           4               6               2               5               11
1           2               1               3               13              14
Average waiting time is:6.000000
Average turnaroundtime is:9.666667
```

Result : Thus the C program to implement the priority scheduling algorithm was executed successfully

Assessment of an Individual experiment	
Algorithm	
Program	
Output	
Viva	

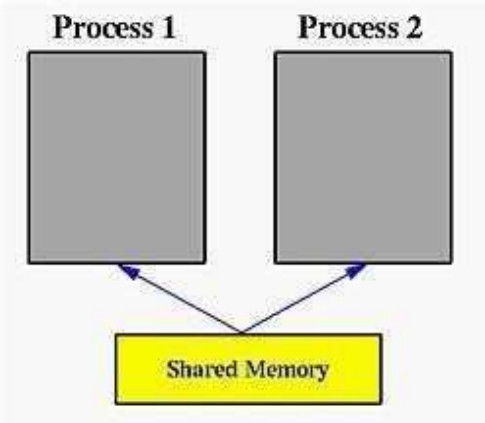
Ex.No : 4 **SIMULATATION of IPC in UNIX**

AIM:

To write a C program to implement inter process communication.

DESCRIPTION:

In the discussion of the **fork()** system call, we mentioned that a parent and its children have separate address spaces. While this would provide a more secured way of executing parent and children processes (because they will not interfere each other), they shared nothing and have no way to communicate with each other. A ***shared memory*** is an extra piece of memory that is *attached* to some address spaces for their owners to use. As a result, all of these processes share the same memory segment and have access to it. Consequently, race conditions may occur if memory accesses are not handled properly. The following figure shows two processes and their address spaces. The yellow rectangle is a shared memory attached to both address spaces and both process 1 and process 2 can have access to this shared memory as if the shared memory is part of its own address space. In some sense, the original address spaces is "extended" by attaching this shared memory.



Shared memory is a feature supported by UNIX System V, including Linux, SunOS and Solaris. One process must explicitly ask for an area, using a ***key***, to be shared by other processes. This process will be called the *server*. All other processes, the *clients*, that know the shared area can access it. However, there is no protection to a shared memory and any process that knows it can access it freely. To protect a shared memory from being accessed at the same time by several processes, a synchronization protocol must be setup.

A shared memory segment is identified by a unique integer, the ***shared memory ID***. The shared memory itself is described by a structure of type **shmid_ds** in header file **sys/shm.h**. To use this file, files **sys/types.h** and **sys/ipc.h** must be included.

AIM: To write a C program to implement inter process communication SHARED MEMORY FOR

WRITER PROCESS

ALGORITHM:

1. Writer requests the entry to critical section.

2. If allowed i.e. wait() gives a true value, it enters and performs the write. If not allowed, it keeps on waiting.

3. It exits the critical section.

```
do {

// writer requests for critical section wait(wrt);

// performs the write

 // leaves the critical section signal(wrt);

 }

while(true);
```

Program:

```
#include <stdio.h>

int main()

{

        // ftok to generate unique key

        key_t key = ftok("shmfile",65);

        // shmget returns an identifier in shmid

        int shmid = shmget(key,1024,0666|IPC_CREAT);
```

```
    // shmat to attach to shared memory
    char str = (char) shmat(shmid,(void*)0,0);
    cout<<"Write Data : ";
    gets(str);
    printf("Data written in memory: %s\n",str);
    //detach from shared memory
    shmdt(str);
    return 0;
}
```

EXECUTION & OUTPUT:



SHARED MEMORY FOR READER PROCESS

ALGORITHM:

1)Reader requests the entry to critical section.

2)If allowed:

2.1) it increments the count of number of readers inside the critical section. If this reader is the first reader entering, it locks the wrt semaphore to restrict the entry of writers if any reader is inside.

2.2)It then, signals mutex as any other reader is allowed to enter while others are already reading.

2.3) After performing reading, it exits the critical section. When exiting, it checks if no more reader is inside, it signals the semaphore wrt as now, writer can enter the critical section.

3)If not allowed, it keeps on waiting.

do {

// Reader wants to enter the critical section

wait(mutex);

// The number of readers has now increased by 1 readcnt++;

// there is atleast one reader in the critical section

// this ensure no writer can enter if there is even one reader

// thus we give preference to readers here if (readcnt==1) wait(wrt);

// other readers can enter while this current reader is inside

// the critical section signal(mutex);

// current reader performs reading here wait(mutex);

// a reader wants to leave readcnt--;

// that is, no reader is left in the critical section, if (readcnt == 0) signal(wrt); // writers can enter signal(mutex);

// reader leaves } while(true);

PROGRAM:

```
#include <stdio.h>

int main()

{

        // ftok to generate unique key

        key_t key = ftok("shmfile",65);


        // shmget returns an identifier in shmid

        int shmid = shmget(key,1024,0666|IPC_CREAT);

        // shmat to attach to shared memory

        char str = (char) shmat(shmid,(void*)0,0);

        printf("Data read from memory: %s\n",str);

        //detach from shared memory

        shmdt(str);

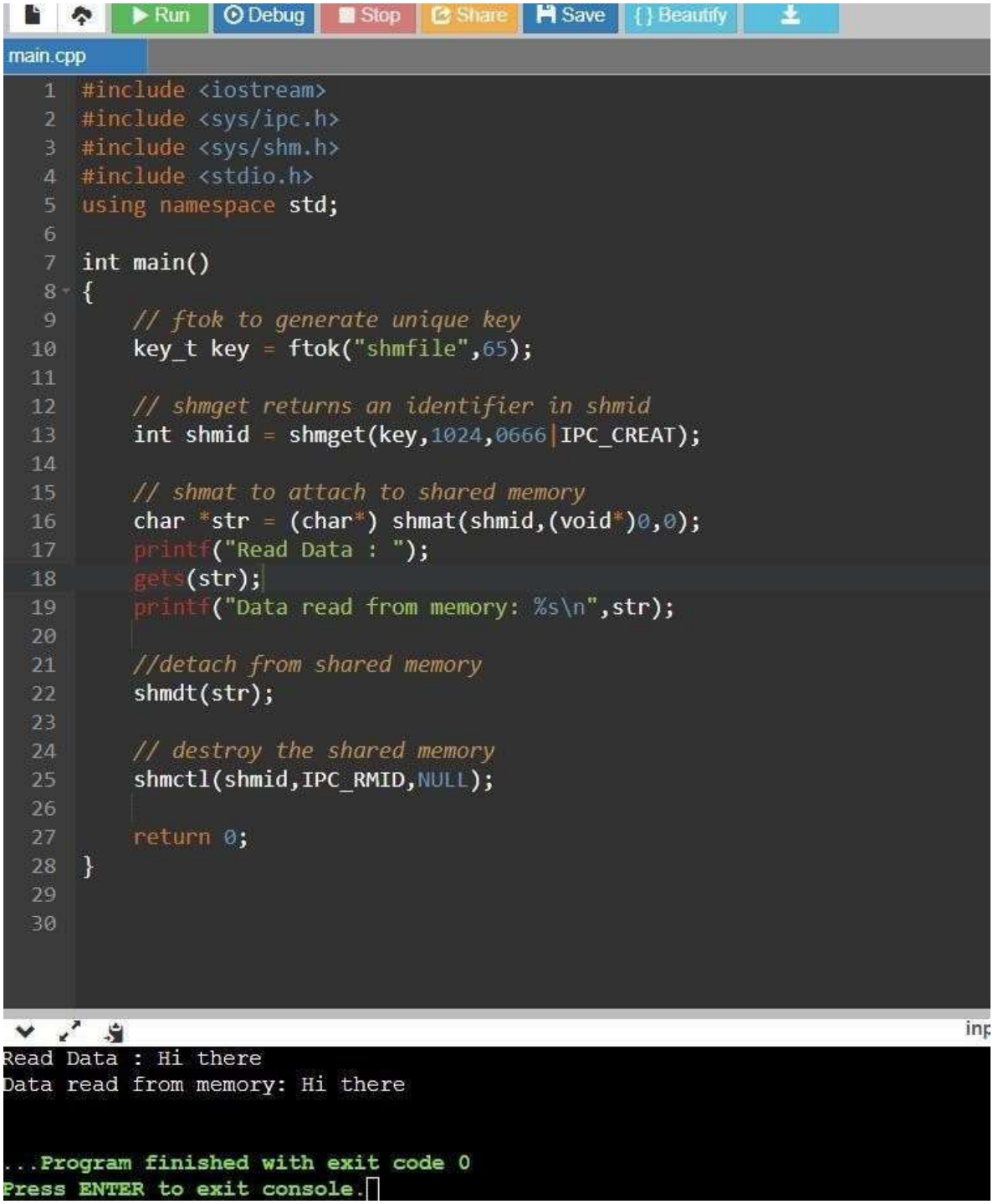
        // destroy the shared memory

        shmctl(shmid,IPC_RMID,NULL);

        return 0;

}
```

EXECUTION & OUTPUT:



Result : To write a C program to implement inter process communication was implemented successfully.

Assessment of an Individual experiment	
Algorithm	
Program	
Output	
Viva	

Ex.No: 5 IMPLEMENTATION OF DEADLOCK - BANKERS ALGORITHM

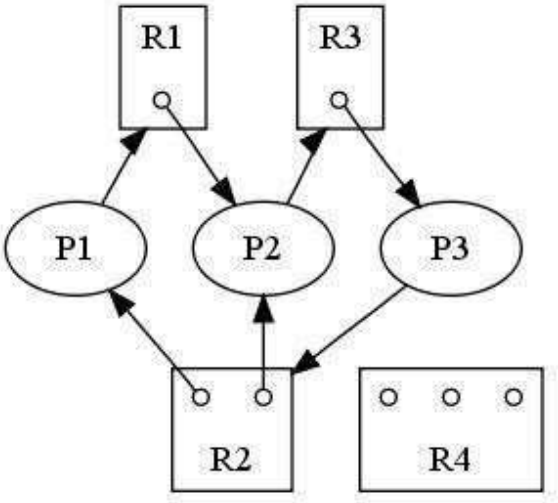
AIM:

To write a UNIX C Program for the Implementation of Banker's

PROBLEM DESCRIPTION:

The Banker's Algorithm was designed and developed by a Dutch Computer Scientist, Edsger Djikstra. The Banker's Algorithm is a Resource Allocation and a Deadlock Avoidance Algorithm.

This algorithm takes analogy of an actual bank where clients request to withdraw cash. The Banking Authorities have some data according to which the cash is lent to the client. The Banker cannot give more cash than the client's request and the total cash available in the bank.



The Banker's Algorithm is divided into two parts:

1. Safety Test Algorithm: This algorithm checks the current state of the system to maintain its Safe State.

2. Resource Request Handling Algorithm: This algorithm verifies if the requested resources, after their allocation to the processes affects the Safe State of the System. If it does, then the request of the process for the resource is denied, thereby maintaining the Safe State.

A State is considered to be Safe if it is possible for all the Processes to Complete its Execution without causing any Deadlocks. An Unsafe State is the one in which the Processes cannot complete its execution.

ALGORITHM:

Step-1: Start the program.

Step-2: Declare the memory for the process.

Step-3: Read the number of process, resources, allocation matrix and available matrix.

Step-4: Compare each and every process using the banker‟s algorithm.

Step-5: If the process is in safe state then it is a not a deadlock process otherwise it is a

deadlock process

Step-6: produce the result of state of process

Step-7: Stop the program

PROGRAM:

```
#include <stdio.h>

int main()

{

   // P0, P1, P2, P3, P4 are the Process names here

   int n, m, i, j, k;

   n = 5; // Number of processes

   m = 3; // Number of resources

   int alloc[5][3] = { { 0, 1, 0 }, // P0     // Allocation Matrix

                       { 2, 0, 0 }, // P1

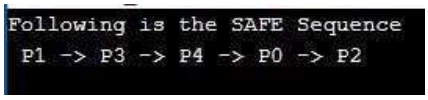
                       { 3, 0, 2 }, // P2

                       { 2, 1, 1 }, // P3
```

```
                { 0, 0, 2 } }; // P4
int max[5][3] = { { 7, 5, 3 }, // P0    // MAX Matrix
                { 3, 2, 2 }, // P1
                { 9, 0, 2 }, // P2
                { 2, 2, 2 }, // P3
                { 4, 3, 3 } }; // P4
int avail[3] = { 3, 3, 2 }; // Available Resources
int f[n], ans[n], ind = 0;
for (k = 0; k < n; k++) {
   f[k] = 0;
}
int need[n][m];
for (i = 0; i < n; i++) {
   for (j = 0; j < m; j++)
      need[i][j] = max[i][j] - alloc[i][j];
}
int y = 0;
for (k = 0; k < 5; k++) {
   for (i = 0; i < n; i++) {
      if (f[i] == 0) {
         int flag = 0;
          for (j = 0; j < m; j++) {
             if (need[i][j] > avail[j]){
                flag = 1;
```

```
                break;
            } }
            if (flag == 0) {
                ans[ind++] = i;
                for (y = 0; y < m; y++)
                    avail[y] += alloc[i][y];
                f[i] = 1;
            } } } }
    printf("Following is the SAFE Sequence\n");
    for (i = 0; i < n - 1; i++)
        printf(" P%d ->", ans[i]);
    printf(" P%d", ans[n - 1]);
    return (0);
}
```

EXECUTION & OUTPUT:



Result: Thus the Banker's algorithm was implemented successfully.

Assessment of an Individual experiment	
Algorithm	
Program	
Output	
Viva	

Ex. No.:6 **PAGE REPLACEMENT ALGORITHMS**

AIM:

To write a program in C to implement page replacement algorithm FIFO

PROBLEM DESCRIPTION:

Page replacement algorithms are used to decide what pages to page out when a page needs to be allocated. This happens when a page fault occurs and free page cannot be used to satisfy allocation

LRU:

"Replace *the page that had not been used* for a longer sequence of time". The frames are empty in the beginning and initially no page fault occurs so it is set to zero. When a page fault occurs the page reference sting is brought into the memory. The operating system keeps track of all pages in the memory, thereby keeping track of the page that had not been used for longer sequence of time. If the page in the page reference string is not in memory, the page fault is incremented and the page that had not been used for a longer sequence of time is replaced. If the page in the page reference string is in the memory take the next page without calculating the next page. Take the next page in the page reference string and check if the page is already present in the memory or not. Repeat the process until all pages are referred and calculate the page fault for all those pages in the page references string for the number of available frames.

AIM : To write c program to implement LRU algorithm

Algorithm

- Declare the size.
- Get the number of pages to be inserted.
- Get the value.
- Declare counter and stack.
- Select the **least recently used** page by counter value.
- Stack them according the selection.
- Display the values.

- Stop the process

Program

```
#include<stdio.h>
int findLRU(int time[], int n){
    int i, minimum = time[0], pos = 0;
    for(i = 1; i < n; ++i){
            if(time[i] < minimum){
                    minimum = time[i];
                    pos = i;
            }
    }
    return pos;
}
int main()
{
   int no_of_frames, no_of_pages, frames[10], pages[30], counter = 0, time[10], flag1, flag2, i, j,
pos, faults = 0;
      printf("Enter number of frames: ");
        scanf("%d", &no_of_frames);
        printf("Enter number of pages: ");
        scanf("%d", &no_of_pages);

    printf("Enter reference string: ");
    for(i = 0; i < no_of_pages; ++i){
    scanf("%d", &pages[i]);
   }
    for(i = 0; i < no_of_frames; ++i){
        frames[i] = -1;
   }
```

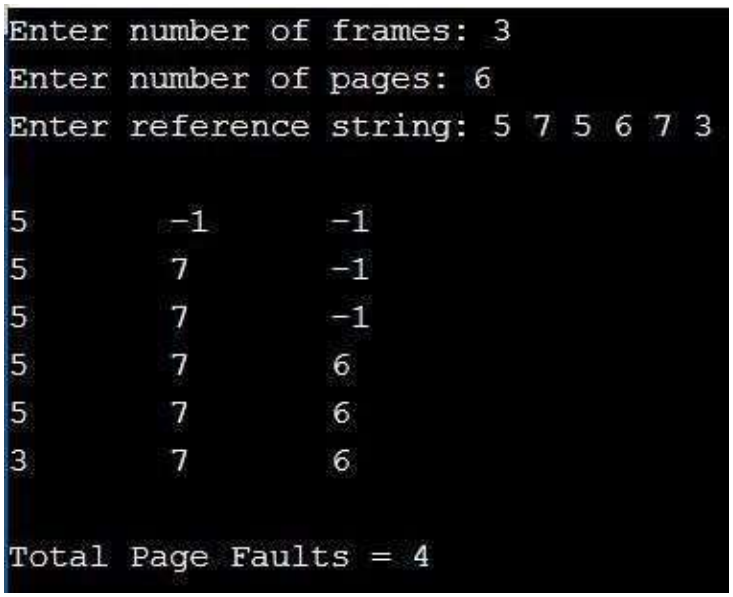
```
for(i = 0; i < no_of_pages; ++i){
   flag1 = flag2 = 0;

  for(j = 0; j < no_of_frames; ++j){
            if(frames[j] == pages[i]){
               counter++;
               time[j] = counter;
                     flag1 = flag2 = 1;
                     break;
            }
  }
  if(flag1 == 0){
     for(j=0;j<no_of_frames;++j){
        if(frames[j]==-1){
            counter++;
            faults++;
            frames[j] = pages[i];
            time[j] = counter;
            flag2 = 1;
            break;
        } } }
  if(flag2 == 0){
            pos = findLRU(time, no_of_frames);
            counter++;
            faults++;
            frames[pos] = pages[i];
            time[pos] = counter;

}
```

```
    printf("\n");
    for(j = 0; j < no_of_frames; ++j){
            printf("%d\t", frames[j]);
    }
    }
    printf("\n\nTotal Page Faults = %d", faults);
  return 0;
}
```

EXECUTION & OUTPUT:



AIM : : To write c program to implement Optimal replacement algorithm

ALGORITHM:

1. Read The Number Of Pages, Reference String, Cache Size
2. Let Cache Size Be N. Add First N Elements From Reference String To Cache Directly. Print Cache And Increment Page Faults.
3. For The Elememts Left Do The Following..
3.1 There May Be 2 Cases. Either The Element In The Reference String May Be In The Cache Already. Or It Is Not Present
3.2 If The Element Is Not In The Cache Do The Following(Uses The Function Ispresent() To Check If The Element Is Present Or Not)
3.2.1 For All The Elements In The Cache Find Respective Length From Current Index In The Reference String (Using Findlength() Function)
3.2.2 Find The Maximum Of The Lengths (Using Findmax() Function)
3.2.3 Replace That Index Of Cache With The Element Of Reference String
3.2.4 Increment Page Fault
3.3 Else Do Nothing
4. Print Page Faults

PROGRAM:

```
#include <stdio.h>

int main()

{

//variable declaration and initialization

int frames_number, pages_number, frames[10],faults, pages[30], temp[10], flag1, flag2, flag3, i, j, k, pos, max, miss = 0;


//code to input the frame number

printf("Enter number of frames: ");

scanf("%d", & frames_number);


//code to input number of pages

printf("Enter number of pages: ");

scanf("%d", &pages_number);


//code to define reference string, page numbers, and frame numbers

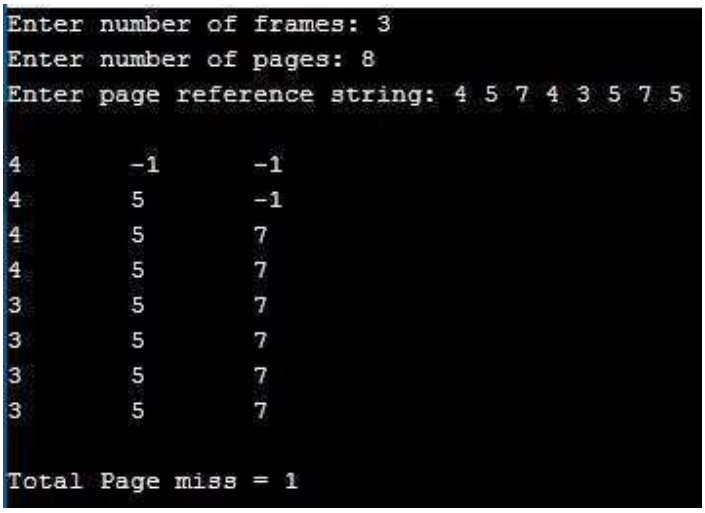
printf("Enter page reference string: ");
```

```
for(i = 0; i < pages_number; ++i){
scanf("%d", &pages[i]);
}
for(i = 0; i < frames_number; ++i){
frames[i] = -1;
}
for(i = 0; i < pages_number; ++i){
flag1 = flag2 = 0;
for(j = 0; j < frames_number; ++j){
if(frames[j] == pages[i]){
flag1 = flag2 = 1;
break;
}
}
//definition of the flag at the starting of the string
if(flag1 == 0){
for(j = 0; j < frames_number; ++j){
if(frames[j] == -1)
{
  faults++;
  frames[j] = pages[i];
  flag2 = 1;
  break;
}
}
}
// definition of the flag at the mid position
```

```
if(flag2 == 0){
flag3 =0;
for(j = 0; j < frames_number; ++j){
temp[j] = -1;
for(k = i + 1; k < pages_number; ++k){
if(frames[j] == pages[k]){
temp[j] = k;
break;
}
}
}
for(j = 0; j < frames_number; ++j){
if(temp[j] == -1){
pos = j;
flag3 = 1;
break;
}
}
//definition of flag at the rear position
if(flag3 ==0){
max = temp[0];
pos = 0;
for(j = 1; j < frames_number; ++j){ if(temp[j] > max){
max = temp[j];
pos = j;
}
}
}
```

```
frames[pos] = pages[i];
miss++;
}
printf("\n");
for(j = 0; j < frames_number; ++j){
printf("%d\t", frames[j]);
}
}
printf("\n\nTotal Page miss = %d", miss);
return 0;
}
```

EXECUTION & OUTPUT:



RESULT : Thus c program to implement Optimal replacement algorithm was implemented successfully.

AIM : To write c program to implement FIFO algorithm

ALGORITHM:

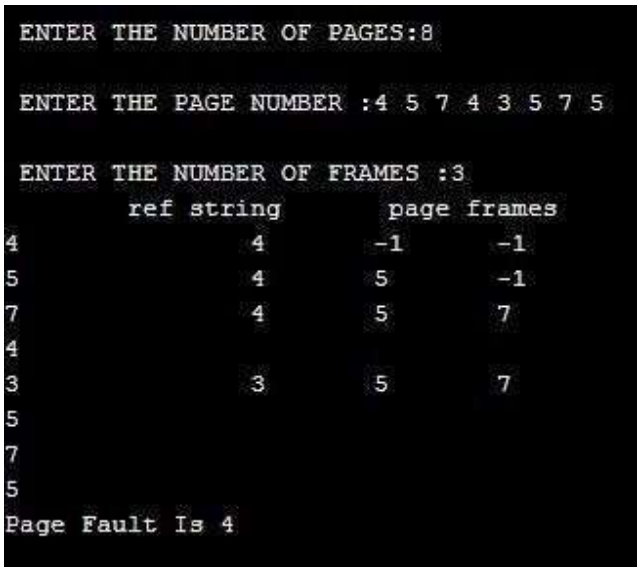
1. Start the process
2. Declare the size with respect to page length
3. Check the need of replacement from the page to memory
4. Check the need of replacement from old page to new page in memory
5. Forma queue to hold all pages
6. Insert the page require memory into the queue
7. Check for bad replacement and page fault
8. Get the number of processes to be inserted
9. Display the values
10. Stop the process

PROGRAM:

```
#include<stdio.h>
int main()
{
int i,j,n,a[50],frame[10],no,k,avail,count=0;
        printf("\n ENTER THE NUMBER OF PAGES:\n");
scanf("%d",&n);
        printf("\n ENTER THE PAGE NUMBER :\n");
        for(i=1;i<=n;i++)
        scanf("%d",&a[i]);
        printf("\n ENTER THE NUMBER OF FRAMES :");
        scanf("%d",&no);
for(i=0;i<no;i++)
        frame[i]= -1;
                j=0;
```

```
                printf("\tref string\t page frames\n");
for(i=1;i<=n;i++){
                        printf("%d\t\t",a[i]);
                        avail=0;
                        for(k=0;k<no;k++)
if(frame[k]==a[i])
                                avail=1;
                        if (avail==0)
                        {
                                frame[j]=a[i];
                                j=(j+1)%no;
                                count++;
                                for(k=0;k<no;k++)
                                printf("%d\t",frame[k]);
}
                        printf("\n");
}
                printf("Page Fault Is %d",count);
                return 0;
}
```

EXECUTION & OUTPUT:



Result : Thus c program to implement FIFO algorithm was implemented successfully.

Assessment of an Individual experiment	
Algorithm	
Program	
Output	
Viva	

MEMORY MANAGEMENT FUNCTIONS

Ex. No. 7a **Memory Management Scheme using Best Fit.**

AIM:

To write a C program to implement the memory management scheme using best fit.

ALGORITHM:

1. Get no. of Processes and no. of blocks.
2. After that get the size of each block and process requests.
3. Then select the best memory block that can be allocated using the above definition.
4. Display the processes with the blocks that are allocated to a respective process.
5. Value of Fragmentation is optional to display to keep track of wasted memory.
6. Stop.

PROGRAM:

```
#include<stdio.h>


int main()
{
    int fragments[10], block[10], file[10], m, n, number_of_blocks, number_of_files, temp, lowest
= 10000;
    static int block_arr[10], file_arr[10];
    printf("\nEnter the Total Number of Blocks:\t");
    scanf("%d", &number_of_blocks);
    printf("\nEnter the Total Number of Files:\t");
    scanf("%d",  &number_of_files);
    printf("\nEnter the Size of the Blocks:\n");
    for(m = 0; m < number_of_blocks; m++)
    {
```

```
    printf("Block No.[%d]:\t", m + 1);
    scanf("%d", &block[m]);
}
printf("Enter the Size of the Files:\n");
for(m = 0; m < number_of_files; m++)
{
    printf("File No.[%d]:\t", m + 1);
    scanf("%d", &file[m]);
}
for(m = 0; m < number_of_files; m++)
{
    for(n = 0; n < number_of_blocks; n++)
    {
        if(block_arr[n] != 1)
        {
            temp = block[n] - file[m];
            if(temp >= 0)
            {
                if(lowest > temp)
                {
                    file_arr[m] = n;
                    lowest = temp;
                }
            }
        }
        fragments[m] = lowest;
        block_arr[file_arr[m]] = 1;
        lowest = 10000;
```

```
        }
    }
    printf("\nFile Number\tFile Size\tBlock Number\tBlock Size\tFragment");
    for(m = 0; m < number_of_files && file_arr[m] != 0; m++)
    {
        printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d", m, file[m], file_arr[m], block[file_arr[m]],
fragments[m]);
    }
printf("\n");
    return 0;
}
```

EXECUTION & OUTPUT:

```
Enter the Total Number of Blocks:       5

Enter the Total Number of Files:        4

Enter the Size of the Blocks:
Block No.[1]:   5
Block No.[2]:   4
Block No.[3]:   3
Block No.[4]:   6
Block No.[5]:   7
Enter the Size of the Files:
File No.[1]:    1
File No.[2]:    3
File No.[3]:    5
File No.[4]:    3

File Number     File Size       Block Number    Block Size      Fragment
0               1               4               7               6
```

Result : Thus c program to implement best fit was executed successfully.

Assessment of an Individual experiment	
Algorithm	
Program	
Output	
Viva	

Ex. No. 7b **Memory Management Scheme using First Fit.**

AIM:

To write a C program to implement the memory management scheme using first fit

ALGORITHM:

1. Get no. of Processes and no. of blocks.
2. After that get the size of each block and process requests.

3.Now allocate processes

if(block size >= process size)

//allocate the process

else

//move on to next block

4. Display the processes with the blocks that are allocated to a respective process.

5.Stop.

PROGRAM:

```
#include<stdio.h>
int main()
{
    static int block_arr[10], file_arr[10];
    int fragments[10], blocks[10], files[10];
    int m, n, number_of_blocks, number_of_files, temp;
    printf("\nEnter the Total Number of Blocks:\t");
    scanf("%d", &number_of_blocks);
    printf("Enter the Total Number of Files:\t");
    scanf("%d", &number_of_files);
    printf("\nEnter the Size of the Blocks:\n");
    for(m = 0; m < number_of_blocks; m++)
    {
```

```
    printf("Block No.[%d]:\t", m + 1);
    scanf("%d", &blocks[m]);
}
printf("Enter the Size of the Files:\n");
for(m = 0; m < number_of_files; m++)
{
    printf("File No.[%d]:\t", m + 1);
    scanf("%d", &files[m]);
}
for(m = 0; m < number_of_files; m++)
{
    for(n = 0; n < number_of_blocks; n++)
    {
        if(block_arr[n] != 1)
        {
            temp = blocks[n] - files[m];
            if(temp >= 0)
            {
                file_arr[m] = n;
                break;
            }
        }
    }
    fragments[m] = temp;
    block_arr[file_arr[m]] = 1;
}
printf("\nFile Number\tBlock Number\tFile Size\tBlock Size\tFragment");
for(m = 0; m < number_of_files; m++)
```

```
    {
        printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d", m, file_arr[m], files[m], blocks[file_arr[m]],
fragments[m]);
    }
    printf("\n");
    return 0;
}
```

EXECUTION & OUTPUT:

```
Enter the Total Number of Blocks:       5
Enter the Total Number of Files:        4

Enter the Size of the Blocks:
Block No.[1]:   5
Block No.[2]:   4
Block No.[3]:   3
Block No.[4]:   6
Block No.[5]:   7
Enter the Size of the Files:
File No.[1]:    1
File No.[2]:    3
File No.[3]:    5
File No.[4]:    3

File Number     Block Number    File Size       Block Size      Fragment
0               0               1               5               4
1               1               3               4               1
2               3               5               6               1
3               2               3               3               0
```

Result : Thus c program to implement best fit was implemented successfully.

Assessment of an Individual experiment	
Algorithm	
Program	
Output	
Viva	

Ex. No. 7c Memory Management Scheme using Worst Fit.

AIM:

To write a C program to implement the memory management scheme using worst fit.

Exercise:

Given memory partition of 100 KB, 500 KB, 200 KB, 300 KB and 600 KB (in order), how would each of the first-fit, best fit and worst fit algorithms place processes of 212 KB, 417 KB, 112 KB and 426 KB (in order)? Which algorithm makes the most efficient use of memory? Either it is worst fit.

ALGORITHM:

Step 1:Input memory block with a size.

Step 2:Input process with size.

Step 3:Initialize by selecting each process to find the maximum block size that can be assigned to the current process.

Step 4:If the condition does not fulfill, they leave the process.

Step 5:If the condition is not fulfilled, then leave the process and check for the next process.

Step 6:Stop.

PROGRAM:

```
#include<stdio.h>
int main()
{
    int fragments[10], blocks[10], files[10];
    int m, n, number_of_blocks, number_of_files, temp, top = 0;
    static int block_arr[10], file_arr[10];
    printf("\nEnter the Total Number of Blocks:\t");
    scanf("%d",&number_of_blocks);
    printf("Enter the Total Number of Files:\t");
    scanf("%d",&number_of_files);
    printf("\nEnter the Size of the Blocks:\n");
```

```
for(m = 0; m < number_of_blocks; m++)
{
    printf("Block No.[%d]:\t", m + 1);
    scanf("%d", &blocks[m]);
}
printf("Enter the Size of the Files:\n");
for(m = 0; m < number_of_files; m++)
{
    printf("File No.[%d]:\t", m + 1);
    scanf("%d", &files[m]);
}
for(m = 0; m < number_of_files; m++)
{
    for(n = 0; n < number_of_blocks; n++)
    {
        if(block_arr[n] != 1)
        {
            temp = blocks[n] - files[m];
            if(temp >= 0)
            {
                if(top < temp)
                {
                    file_arr[m] = n;
                    top = temp;
                }
            }
        }
        fragments[m] = top;
```

```
            block_arr[file_arr[m]] = 1;
            top = 0;
        }
    }
    printf("\nFile Number\tFile Size\tBlock Number\tBlock Size\tFragment");
    for(m = 0; m < number_of_files; m++)
    {
        printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d", m, files[m], file_arr[m], blocks[file_arr[m]],
fragments[m]);
    }
    printf("\n");
    return 0;
}
```

EXECUTION & OUTPUT:

```
Enter the Total Number of Blocks:       5
Enter the Total Number of Files:        4

Enter the Size of the Blocks:
Block No.[1]:   5
Block No.[2]:   4
Block No.[3]:   3
Block No.[4]:   6
Block No.[5]:   7
Enter the Size of the Files:
File No.[1]:    1
File No.[2]:    3
File No.[3]:    5
File No.[4]:    3

File Number     File Size       Block Number    Block Size      Fragment
0               1               4               7               6
1               3               0               5               0
2               5               0               5               0
3               3               0               5               0
```

Result : Thus c program for worst fit was executed successfully

Assessment of an Individual experiment	
Algorithm	
Program	
Output	
Viva	

Ex No: 8 IMPLEMENTATION OF DISK SCHEDULING ALGORITHMS

AIM:

To write a 'C' program to implement the Disk Scheduling algorithm for First Come First Served (FCFS), Shortest Seek Time First (SSTF), and SCAN.

PROBLEM DESCRIPTION:

Disk Scheduling is the process of deciding which of the cylinder request is in the ready queue is to be accessed next.

The access time and the bandwidth can be improved by scheduling the servicing of disk I/O requests in good order.

Access Time:

The access time has two major components: **Seek time and Rotational Latency.**

Seek Time:

Seek time is the time for disk arm to move the heads to the cylinder containing the desired sector.

Rotational Latency:

Rotational latency is the additional time waiting for the disk to rotate the desired sector to the disk head.

Bandwidth:

The disk bandwidth is the total number of bytes transferred, divided by the total time between the first request for service and the completion of the last transfer.

PROCEDURE:

1. Input the maximum number of cylinders and work queue and its head starting position.
2. **First Come First Serve Scheduling (FCFS) algorithm –** The operations are performed in order requested
3. There is no reordering of work queue.
4. Every request is serviced, so there is no starvation
5. The seek time is calculated.

6. **Shortest Seek Time First Scheduling (SSTF) algorithm –** This algorithm selects the request with the minimum seek time from the current head position.

7. Since seek time increases with the number of cylinders traversed by the head, SSTF chooses the pending request closest to the current head position.

8. The seek time is calculated.

9. **SCAN Scheduling algorithm –**The disk arm starts at one end of the disk, and moves toward the other end, servicing requests as it reaches each cylinder, until it gets to the other end of the disk.

10. At the other end, the direction of head movement is reversed and servicing continues.

11. The head continuously scans back and forth across the disk.

12. The seek time is calculated.

13. Display the seek time and terminate the program.

AIM: To write a C program to implement the Disk Scheduling algorithm for First Come First Served (FCFS), Shortest Seek Time First (SSTF), and SCAN.

ALGORITHM:

1. Let Request array represents an array storing indexes of tracks that have been requested in ascending order of their time of arrival. head is the position of disk head.

2. Let us one by one take the tracks in default order and calculate the absolute distance of the track from the head.

3. Increment the total seek count with this distance.

4. Currently serviced track position now becomes the new head position.

5.Go to step 2 until all tracks in request array have not been serviced.

Program :

```
#include<stdio.h>

int main()

{

    int queue[20],n,head,i,j,k,seek=0,max,diff;

    float avg;
```

```
        printf("Enter the max range of disk:");
        scanf("%d",&max);
        printf("Enter the size of queue request:");
        scanf("%d",&n);
        printf("Enter the queue of disk positions to be read:");
        for(i=1;i<=n;i++)
        scanf("%d",&queue[i]);
        printf("Enter the initial head position:");
        scanf("%d",&head);
        queue[0]=head;
        for(j=0;j<=n-1;j++)
        {
                diff=abs(queue[j+1]-queue[j]);
                seek+=diff;
                printf("Disk head moves from %d to %d with seek :%d\n",queue[j],queue[j+1],diff);
        }
        printf("Total seek time is %d\n",seek);
        avg=seek/(float)n;
        printf("Average seek time is %f\n",avg);
        return 0;
}
```

EXECUTION & OUTPUT:

```
Enter the max range of disk:200
Enter the size of queue request:8
Enter the queue of disk positions to be read:90 120 35 122 38 128 65 68
Enter the initial head position:50
Disk head moves from 50 to 90 with seek :40
Disk head moves from 90 to 120 with seek :30
Disk head moves from 120 to 35 with seek :85
Disk head moves from 35 to 122 with seek :87
Disk head moves from 122 to 38 with seek :84
Disk head moves from 38 to 128 with seek :90
Disk head moves from 128 to 65 with seek :63
Disk head moves from 65 to 68 with seek :3
Total seek time is 482
Average seek time is 60.250000
```

SSTF:

ALORITHM:

1. Let Request array represents an array storing indexes of tracks that have been requested. head is the position of disk head.

2. Find the positive distance of all tracks in the request array from head.

3. Find a track from requested array which has not been accessed/serviced yet and has minimum distance from head.

4. Increment the total seek count with this distance.

5. Currently serviced track position now becomes the new head position.

6.Go to step 2 until all tracks in request array have not been serviced

PROGRAM:

```
#include<stdio.h>

#include<conio.h>

#include<math.h>

int main()

{

int queue[100],t[100],head,seek=0,n,i,j,temp;

float avg;
```

```
// clrscr();
printf("* SSTF Disk Scheduling Algorithm *\n");
printf("Enter the size of Queue\t");
scanf("%d",&n);
printf("Enter the Queue\t");
for(i=0;i<n;i++)
{
scanf("%d",&queue[i]);
}
printf("Enter the initial head position\t");
scanf("%d",&head);
for(i=1;i<n;i++)
t[i]=abs(head-queue[i]);
for(i=0;i<n;i++)
{
for(j=i+1;j<n;j++)
{
if(t[i]>t[j])
{
temp=t[i];
t[i]=t[j];
t[j]=temp;
temp=queue[i];
queue[i]=queue[j];
queue[j]=temp;
}
```

```
}

}

for(i=1;i<n-1;i++)

{

seek=seek+abs(head-queue[i]);

head=queue[i];

}

printf("\nTotal Seek Time is%d\t",seek);

avg=seek/(float)n;

printf("\nAverage Seek Time is %f\t",avg);

return 0;

}
```

EXECUTION & OUTPUT:

```
* SSTF Disk Scheduling Algorithm *
Enter the size of Queue:10
Enter the Queue:23 12 9 56 76 45 34 5 8 10
Enter the initial head position:90

Total Seek Time is85
Average Seek Time is 8.500000
```

SCAN:

ALGORITHM:

1. Let Request array represents an array storing indexes of tracks that have been requested in ascending order of their time of arrival. head is the position of disk head.

2. Let direction represents whether the head is moving towards left or right.

3.In the direction in which head is moving service all tracks one by one.

4.Calculate the absolute distance of the track from the head.

5. Increment the total seek count with this distance.

6. Currently serviced track position now becomes the new head position.

7,Go to step 3 until we reach at one of the ends of the disk.

If we reach at the end of the disk reverse the direction and go to step 2 until all tracks in request array have not been serviced.

PROGRAM:

```
#include<stdio.h>

#include<math.h>

int main()


{

int queue[20],n,head,i,j,k,seek=0,max,diff,temp,queue1[20],queue2[20],

temp1=0,temp2=0;

float avg;

printf("Enter the max range of disk\n");

scanf("%d",&max);

printf("Enter the initial head position\n");

scanf("%d",&head);

printf("Enter the size of queue request\n");

scanf("%d",&n);

printf("Enter the queue of disk positions to be read\n");

for(i=1;i<=n;i++)

{

scanf("%d",&temp);

if(temp>=head)

{

queue1[temp1]=temp;

temp1++;
```

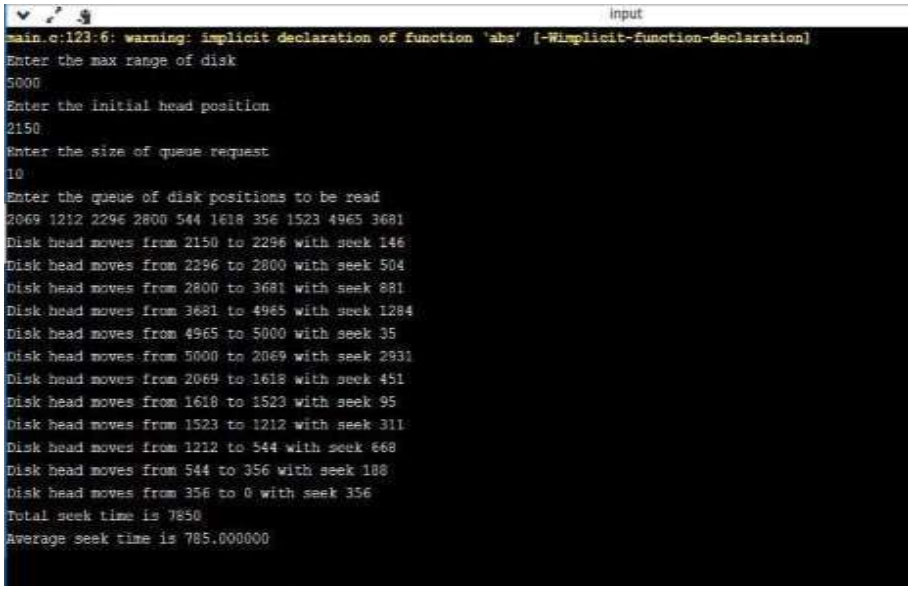
```
}
else
{
queue2[temp2]=temp;
temp2++;
}
}

for(i=0;i<temp1-1;i++)
{
for(j=i+1;j<temp1;j++)
{
if(queue1[i]>queue1[j])
{
temp=queue1[i];
queue1[i]=queue1[j];
queue1[j]=temp;
}
}
}
for(i=0;i<temp2-1;i++)
{
for(j=i+1;j<temp2;j++)
{
if(queue2[i]<queue2[j])
{
```

```
temp=queue2[i];
queue2[i]=queue2[j];
queue2[j]=temp;
}
}
}
for(i=1,j=0;j<temp1;i++,j++)

queue[i]=queue1[j];
queue[i]=max;
for(i=temp1+2,j=0;j<temp2;i++,j++)
queue[i]=queue2[j];
queue[i]=0;
queue[0]=head;
for(j=0;j<=n+1;j++)
{
diff=abs(queue[j+1]-queue[j]);
seek+=diff;
printf("Disk head moves from %d to %d with seek %d\n",queue[j],queue[j+1],diff);
}
printf("Total seek time is %d\n",seek);
avg=seek/(float)n;
printf("Average seek time is %f\n",avg);
return 0;
}
```

EXECUTION & OUTPUT:



Result : Thus implemented disk scheduling algorithms.

Assessment of an Individual experiment	
Algorithm	
Program	
Output	
Viva	

Ex No: 9 IMPLEMENTATION OF ACCESS CONTROL MECHANISM

AIM:

To write a 'C' program to implement various Access Control Mechanism in Operating Systems.

Access control is the traditional center of gravity of computer security. It is where security engineering meets computer science. Its function is to control which principals (persons, processes, machines, . . .) have access to which resources in the system—which files they can read, which programs they can execute, how they share data with other principals, and so on.

The access control mechanisms, which the user sees at the application level, may express a very rich and complex security policy. A modern online business could assign staff to one of dozens of different roles, each of which could initiate some subset of several hundred possible transactions in the system. Some of these (such as credit card transactions with customers) might require online authorization from a third party while others (such as refunds) might require dual control.The applications may be written on top of middleware, such as a database management system or bookkeeping package, which enforces a number of protection properties. For example, bookkeeping software may ensure that a transaction that debits one ledger for a certain amount must credit another ledger for the same amount

The middleware will use facilities provided by the underlying operating system. As this constructs resources such as files and communications ports from lower-level components, it acquires the responsibility for providing ways to control access to them.

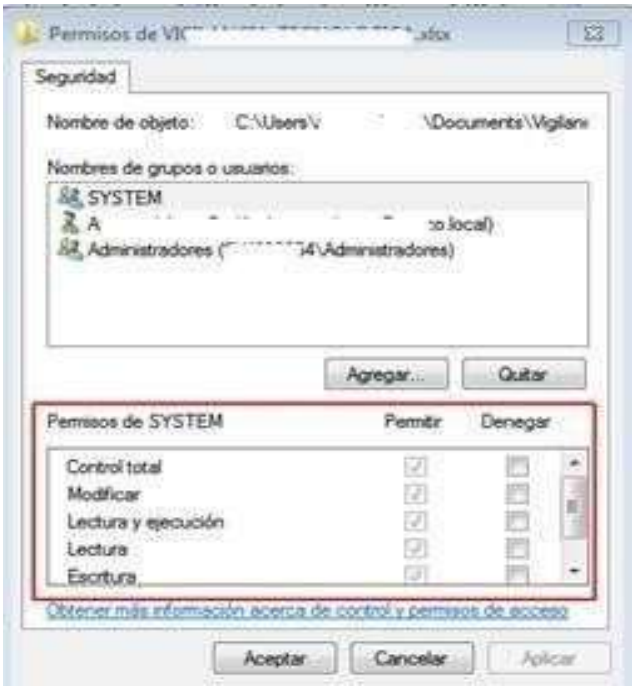
Finally, the operating system access controls will usually rely on hardware features provided by the processor or by associated memory management hardware. These control which memory addresses a given process can access

Types of Access Control.

Based on the restrictions and access control to systems, we come across three main types: discretionary access control (DAC), role based access (RBAC) and mandatory access control (MAC).

Discretionary Access control, DAC: Discretionary access control is a means of restricting access to objects based on the identity of subjects that try to operate or access them. The most representative example is the mechanism of permissions established by the owner (user/group) of the subject. Therefore, it's the owner of the object who determines which users and with which privileges they access his/her objects. This access mechanism is present in the majority of operating systems. In this type of access control its normal to use attributes (read, write, execute,

etc) to mark the permissions applied to the object. In the same way, there is an owner/user and a set of options that can be shared (groups, other users…).



DAC permissions in a Windows file -

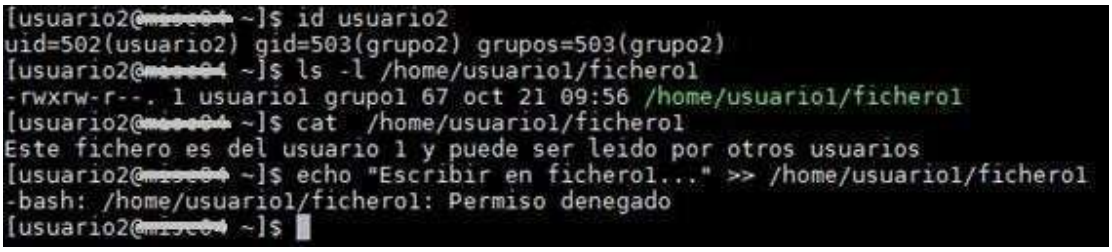
In the particular case of UNIX/Linux systems, the objects of the system (files, directories, etc) include three attributes: read (r), write (w) and execute (x). These permissions are assigned by the user for the group and others (users that are not owners and don't belong the group).

In Linux, with the ls -l command we can visualize the basic DAC permissions. In the following example we can see a system, where user1, who belongs to group1, has fixed the read/write/execute permissions **(rwx)** to himself as the owner. The users that belong to the same group (grupo1) have the same read and write permissions **(rw-)** and any other user only has a read permission **(r--)**:

```
[usuario1@ ~]$ ls -l
total 0
-rwxrw-r--. 1 usuario1 grupo1 0 oct 21 09:45 fichero1
[usuario1@ ~]$
```

*Discretional access permissions (DAC), traditional in *NIX systems -*

This way, any read, write or execute action that doesn't comply with these permission will be denied.



- usuario2 can read but not modify usuario1's fichero1 because of the assigned permissions -

Other properties exist such as ACLs (access control list) or other a href="http://en.wikipedia.org/wiki/File_system_permissions" rel="external">special permissions such as sticky bit, or setuid, setgid (Linux) permissions that add more options to the DAC but are out of the introductory reach of this part.

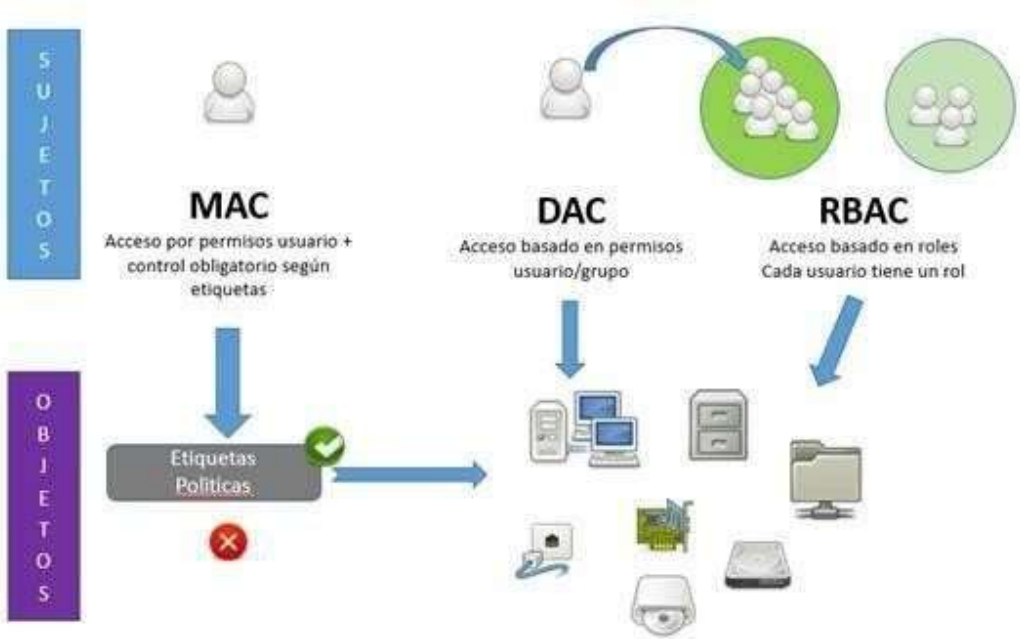
Role-Based Access Control (RBAC)

Discretional access controls don't provide a sufficient granularity to enable a more defined and structured segmentation in a complex system with multiple users and functions. In this case, a role mechanism offers greater versatility. Role-based access control consists in the definition of roles that have been attributed a number of characteristics applied to the permissions and actions that they can carry out, including controlling other roles. It is, in a way, a hierarchical system of classes. Often used in organizations with a great number of users where different work groups or departments with different functions are integrated, such as for example systems, development, commercial, general service departments. With this mechanism, access to objects and tasks can be efficiently segmented and organized. Notable cases of these mechanisms are LDAP, Active Directory of Microsoft Windows or *FreeIPA* of Fedora/Redhat. Some UNIX systems such as *Solaris* or *AIX* all i*mplement this system of privileges.*

Mandatory Access Control, MAC: *This access mechanism is a compliment of the previous ones and adds another safety layer for access and privilege control. MAC bases itself on "tagging" every element in the system that will then undergo the access control policies that have been configured. Therefore, in any operation by a subject on an object the tags will be verified and the established MAC policies will be applied to determine if the operation is allowed, even when it has complied with other security controls. In contrast with the discretionary access control, the user can modify permissions and tags but can't fix access controls that suppose a violation of the*

system's MAC policies. It's the responsibility of a privileged user to establish the MAC's central policies that will govern the controls to be applied depending on the established tags. Examples of these control methods are SELinux in Linux/Redhat/Centos/Fedora distributions or AppArmor in Linux SuSE.

.



Types of access control -

Viva-Voce questions:

1. Define Access Control
2. What are the types of Access Control?
3. Why Access Control is needed?
4. Where this access control is implemented?
5. Where this Access Control information is stored?

Experiments addressing COs:

The experiments mentioned above address CO5

Assessment of an Individual experiment	
Algorithm	
Program	
Output	
Viva	
Total	

Result

Access Control Mechanism have been exeuted successfully.

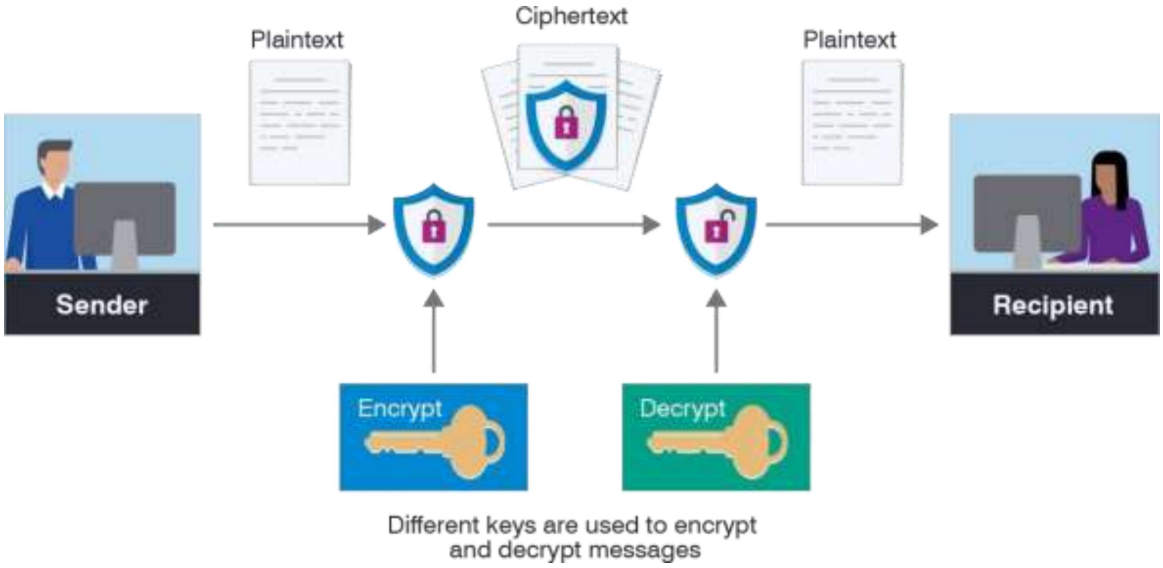
Ex No: 10 IMPLEMENTATION OF CRYPTOGRAPHIC ALGORITHMS

AIM:

To write a 'C' program to implement various Cryptography Algorithms such as RSA, DEA, Blowfish.

Definition

In cryptography, encryption is the process of encoding a message or information in a way that only authorized parties can access it and those who are not authorized cannot.



Encryption Types / Methods

Asymmetric Encryption

In public-key encryption schemes, the encryption key is published for anyone to use and for encrypting messages. Only the receiving party has access to the decryption key that enables messages to be read. Public-key encryption was first described in a secret document in 1973. Before that, all encryption schemes were symmetric-key (also called private-key).

Symmetric Encryption

In symmetric-key schemes, the encryption and decryption keys are the same. Communicating parties must have the same key in order to achieve secure communication.

Encryption Algorithms

Triple DES

Triple DES was designed to replace the original Data Encryption Standard (DES) algorithm, which hackers learned to defeat with ease. At one time, Triple DES was the recommended standard and the most widely used symmetric algorithm in the industry. Triple DES uses three individual keys with 56 bits each. The total key length adds up to 168 bits, but experts say that 112-bits in key strength is more like it.Though it is slowly being phased out, Triple DES is still a dependable hardware encryption solution for financial services and other industries.

RSA

RSA is a public-key encryption algorithm and the standard for encrypting data sent over the internet. It also happens to be one of the methods used in PGP and GPG programs. Unlike Triple DES, RSA is considered an asymmetric encryption algorithm because it uses a pair of keys. The public key is used to encrypt a message and a private key to decrypt it. It takes attackers quite a bit of time and processing power to break this encryption code.

AES

The Advanced Encryption Standard (AES) is the algorithm trusted as the standard by the U.S. government and many other organizations. Although it is extremely efficient in 128-bit form, AES also uses keys of 192 and 256 bits for heavy-duty encryption. AES is considered resistant to all attacks, with the exception of brute-force attacks, which attempt to decipher messages using all possible combinations in the 128-, 192- or 256-bit cipher. Still, security experts believe that AES will eventually become the standard for encrypting data in the private sector.

Encryption Standards

There are a number of standards related to cryptography. Here are the following standards for encryption:

- Data Encryption Standard (now obsolete)
- Advanced Encryption Standard
- RSA (the original public-key algorithm)
- Open PGP

File Encryption Overview

File system-level encryption, often called file and folder encryption, is a form of disk encryption where individual files or directories are encrypted by the file system itself.

Disk Encryption Overview

Disk encryption is a technology that protects information by converting it into unreadable code that cannot be deciphered easily by authorized users. Disk encryption uses disk encryption software or hardware to encrypt every bit of data that goes on a disk or disk volume.

Email Encryption Overview

Email encryption is encryption of email messages designed to protect the content from being read by entities other than the intended recipients. Email encryption may also include authentication. Email is not secure and may disclose sensitive information. Most emails are currently transmitted in the clear (not encrypted) form. By means of some available tools, people other than designated recipients can read the email content. Email encryption traditionally uses one of two protocols, either TLS or end-to-end encryption. Within end-to-end encryption, there are several options, including PGP and S/MIME protocols.

Encryption Best Practices

1. **Know the laws:** When it comes to safeguarding the personally identifiable information, organizations must adhere to many overlapping, privacy-related regulations. The top six regulations that impact many organizations include: FERPA, HIPAA, HITECH, COPPA, PCI DSS and state-specific data breach notifications laws.
2. **Assess the data:** A security rule under HIPAA does not explicitly require encryption, but it does state that entities should perform a data risk assessment and implement encryption if the evaluation indicates that encryption would be a "reasonable and appropriate" safeguard. If an organization decides not to encrypt electronic protected health information (ePHI), the institution must document and justify that decision and then implement an "equivalent alternative measure."
3. **Determine the required or needed level of encryption**: The U.S. Department of Health and Human Services (HHS) turns to the National Institute of Standards and Technology (NIST) for recommended encryption-level practices. HHS and NIST have both produced robust documentation for adhering to HIPAA's Security Rule. NIST Special Publication 800-111 takes a broad approach to encryption on user devices. In a nutshell, it states that when there is even a remote possibility of risk, encryption needs to be in place. FIPS 140-2, which incorporates AES into its protocols, is an ideal choice. FIPS 140-2 helps education entities ensure that PII is "rendered unusable, unreadable or indecipherable to unauthorized individuals." A device that meets FIPS 140-2 requirements has a cryptographic erase function that "leverages the encryption of target data by enabling sanitization of the target data's encryption key, leaving only the cipher text remaining on the media, effectively sanitizing the data."
4. **Be mindful of sensitive data transfers and remote access:** Encryption must extend beyond laptops and backup drives. Communicating or sending data over the internet needs Transport Layer Security (TLS), a protocol for transmitting data over a network, and AES encryption. When an employee accesses an institution's local network, a secure VPN

connection is essential when ePHI is involved. Also, before putting a handful of student files on a physical external device for transfer between systems or offices, the device must be encrypted and meet FIPS 140-2 requirements to avoid potential violations.

5. **Note the fine print details:** Unfortunately, many schools fail to engage in proper due diligence in reviewing third-party services' privacy and data-security policies, and inadvertently authorize data collection and data-mining practices that parents/students find unacceptable or violate FERPA. Regulatory compliance entails much more than simply password-protecting an office's workstations. It requires using encryption to protect data-at-rest when stored on school systems or removable media device. Remember that data at rest that is outside the school's firewall (or "in the wild") is the top source of security breaches.

Aim :

To write c program to implement RSA algorithm

Algorithm:

Working of RSA Algorithm

RSA involves use of public and private key for its operation. The keys are generated using the following steps:-

1.Two prime numbers are selected as p and q

2.n = pq which is the modulus of both the keys.

3.Calculate totient = (p-1)(q-1)

4. Choose e such that e > 1 and coprime to totient which means gcd (e, totient) must be equal to 1, e is the public key

5. Choose d such that it satisfies the equation de = 1 + k (totient), d is the private key not known to everyone.

6. Cipher text is calculated using the equation c = m^e mod n where m is the message.

7. With the help of c and d we decrypt message using equation m = c^d mod n where d is the private key.

Program :

```
#include<stdio.h>
#include<math.h>

//to find gcd
int gcd(int a, int h)
{
    int temp;
    while(1)
    {
        temp = a%h;
        if(temp==0)
        return h;
        a = h;
        h = temp;
    }
}

int main()
{
    //2 random prime numbers
    double p = 3;
    double q = 7;
    double n=p*q;
    double count;
    double totient = (p-1)*(q-1);
```

```
//public key
//e stands for encrypt
double e=2;

//for checking co-prime which satisfies e>1
while(e<totient){
count = gcd(e,totient);
if(count==1)
   break;
else
   e++;
}

//private key
//d stands for decrypt
double d;

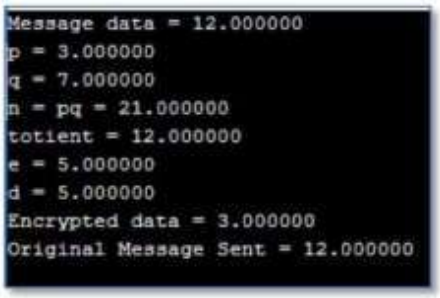
//k can be any arbitrary value
double k = 2;

//choosing d such that it satisfies d*e = 1 + k * totient
d = (1 + (k*totient))/e;
double msg = 12;
double c = pow(msg,e);
double m = pow(c,d);
c=fmod(c,n);
```

```
    m=fmod(m,n);


    printf("Message data = %lf",msg);
    printf("\np = %lf",p);
    printf("\nq = %lf",q);
    printf("\nn = pq = %lf",n);
    printf("\ntotient = %lf",totient);
    printf("\ne = %lf",e);
    printf("\nd = %lf",d);
    printf("\nEncrypted data = %lf",c);
    printf("\nOriginal Message Sent = %lf",m);
    return 0;
}
```

OUTPUT :



DES ALGORITHM

What is DES Encryption Algorithm?

The DES algorithm is also sometimes referred to as Data Encryption Algorithm (DEA). The DES encryption algorithm is a symmetric key algorithm for the encryption of data. The block size is of 64 bits.

The DES is an archetypal block cipher which takes a fixed length string of plain-text bits. There's another improvised version of this algorithm which is Triple DES Algorithm.

The simplified DES (S-DES) is a modified version of the data encryption standard DES algorithm. Another modified version of the DES algorithm is famously known as Triple DES. The key generator method creates 16 48-bit keys

PROGRAM:

```
#include<stdio.h>

int main()

{

    int array_a1[30], array_a2[30], array_a3[30], array_a4[30], array_a5[30], array_a6[30], array_a7[30], array_a8[30];

    int div, count, j, key, m, plaintext, temp, dec = 0;

    printf("\nEnter a Plain-Text value:\t");

    scanf("%d", &plaintext);

    printf("\nEnter the Key:\t");

    scanf("%d", &key);

    printf("\nEnter the Bit-Stream\n");

    for(count = 0; count < plaintext; count++)

    {

        scanf("%d", &array_a1[count]);

    }

    div = plaintext / 2;

    temp = div - key;

    for(count = 0; count <= temp; count++)

    {

        array_a3[count] = 0;

        dec++;

    }

    dec = dec - 1;
```

```
printf("Enter the Key Bit Stream\n");
for(count = 0; count < key; count++)
{
    scanf("%d", &array_a3[dec++]);
}
for(count = 0; count < 2; count++)
{
    printf("%d", array_a8[count]);
}
printf("Left Hand\n");
for(count = 0; count < div; count++)
{
    array_a5[count] = array_a1[count];
    printf("%d", array_a1[count]);
}
printf("Right Hand\n");
for(count = div; count < plaintext; count++)
{
    array_a2[count] = array_a1[count];
    printf("%d", array_a1[count]);
}
for(j = 0, m = div; j < dec, m < plaintext; j++, m++)
{
    if(array_a2[m] == 1 && array_a3[j] == 1)
    {
        array_a6[j] = 0;
```

```
    }
    else if(array_a2[m] == 1 && array_a3[j] == 0)
    {
        array_a6[j] = m;
    }
    else
    {
        array_a6[j] = 0;
    }
}
printf("\nFirst XOR\n");
for(count = 0; count < div; count++)
{
    printf("%d", array_a6[count]);
}
for(j = 0, m = 0; j < div, j++; j++, m++)
{
    if(array_a5[m] = 1 && array_a6[j] == 1)
    {
        array_a4[j] = 0;
    }
    else if(array_a5[m] = 1 && array_a6[j] == 0)
    {
        array_a4[j] = m;
    }
    else if(array_a5[m] == 0 && array_a6[j] == 1)
```

```
        {
            array_a4[j] = 0;
        }
    }
    printf("\nSecond XOR\n");
    for(count = 0; count < div; count++)
    {
        printf("%d", array_a4[j]);
    }
    return 0;
}
```

Viva-Voce questions:

1. What is **cryptography**?
2. What exactly are **encryption** and decryption?
3. What is plaintext or cleartext?
4. What is ciphertext?
5. How does the **encryption** process actually take place?
6. What are the origins of **cryptography**?
7. What is the Caesar **cipher**?
8. What is the goal of **cryptography**?

Experiments addressing COs:

The experiments mentioned above address CO5

Assessment of an Individual experiment	
Algorithm	
Program	
Output	
Viva	
Total	

Result :

To write a 'C' program to implement various Cryptography Algorithms such as RSA, DEA, Blowfish was implemented successfully.